Information Sciences 658 (2024) 120021

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Novel deterministic and probabilistic forecasting methods for crude oil price employing optimized deep learning, statistical and hybrid models

Sourav Kumar Purohit [a,1], Sibarama Panigrahi [b,*]

[a] Department of Computer Science Engineering and Applications, Sambalpur University Institute of Information Technology, Jyoti Vihar, Burla, Odisha 768019, India
[b] Department of Computer Science and Engineering, National Institute of Technology, Rourkela, Odisha, 769008, India

ARTICLE INFO

Keywords:
Crude Oil Price Forecasting
Optimized Deep Learning Model
Optimized Statistical Model
Optimized Hybrid Model
Deterministic Forecasting
Probabilistic Forecasting

ABSTRACT

In this paper, individual and hybrid methods are proposed employing optimized statistical and deep learning (DL) models for deterministic (point) and probabilistic (interval) forecasting of crude oil price time series. The statistical models are optimized using the Forecast package of R. To enhance the performance of DL models, a novel pruning DE-DL method is proposed, which employs the differential evolution (DE) algorithm to optimize architecture and continuous and discrete-valued hyper-parameters. The proposed DE-DL method is so generic that it can be applied to optimize different DL models for any supervised learning problem. Five DL models (LSTM, BiLSTM, GRU, CNN, and ConvLSTM) are optimized for forecasting monthly crude oil prices and hybridized with an optimized ARIMA model for developing optimized additive and multiplicative hybrid forecasting models. The effectiveness of the proposed methods is evaluated through deterministic and probabilistic forecasting measures, comparing the results with six optimized statistical models, thirteen machine learning models, five optimized DL models, and ten optimized hybrid models. It is observed from the simulation results that the proposed optimized Additive-ARIMA-GRU hybrid model provides statistically superior forecasts, and the t Location Scale distribution is more suitable than the Gaussian distribution for computing reliable prediction intervals with different significance levels.

1. Introduction

The crude oil price (COP) is one of the most vital universal macro indicators. Quoted by most news providers worldwide, "it serves as a barometer for economic perspectives, inflation, currencies movement, and political unrest in the Middle East". Despite superior climate goals, extensive usage of renewable energy generation sources, and growth of electric vehicles, crude oil is one of the world's prominent energy sources. It is considered one of the crucial measures of global economic trends, and its price fluctuations significantly impact the economic growth of nations [1]. The price of crude oil varies dynamically and is highly non-linear and irregular. Therefore, forecasting the COP has been a crucial and challenging problem in forecasting research. Fluctuations in COP significantly affect the economy of oil-importing and oil-exporting nations [2]. For example, a fall in COP affects the economic growth of the exporting

* Corresponding author.
E-mail addresses: skpurohit@suiit.ac.in (S.K. Purohit), panigrahis@nitrkl.ac.in (S. Panigrahi).
[1] ORCID: 0000-0002-3343-6818.

https://doi.org/10.1016/j.ins.2023.120021
Received 23 June 2023; Received in revised form 11 November 2023; Accepted 15 December 2023
Available online 17 December 2023

countries, while a rise will cause inflation, leading to a recession in the importing countries [3]. Hence, accurately predicting the COP has become an important issue that has gained remarkable attention from the Governments of different nations as it assists policy makers in making appropriate policies and decisions regarding energy resources [4].

Prediction of COP, of course, is a challenging problem due to its dependence on various factors, including exchange rate, geopolitics, demand and supply, speculative trading, significant unexpected events, and so on. Traditionally, statistical models, including generalized autoregressive conditional heteroskedasticity (GARCH) class models, autoregressive integrated moving average (ARIMA) model, vector autoregressive (VAR) models, and linear regression (LR) model, have been used in COP forecasting. Statistical models like ARIMA assume the time series as an outcome of a linear phenomenon, providing poor forecasts for the complex, nonlinear, and non-Gaussian COP time series. Although the GARCH and VAR models can handle nonlinear patterns, it is difficult to determine the most parsimonious model for the COP time series. As a result, the nonlinear, data-driven, and self-adaptive machine learning (ML) models, particularly deep learning (DL) models, have recently been employed to forecast the COP time series.

Recent literature revealed the efficiency of ML models including DL models like long short-term memory (LSTM) [5–7,9,10,13,16,18], gated recurrent unit (GRU) [6,9,12,14,15], convolutional neural network (CNN) [17], recurrent neural network (RNN) [6,10], deep RNN (DRNN) [11] for forecasting of COP. Despite different studies having used different DL models for COP forecasting, no systematic study has been conducted to evaluate the comparative performance of different DL models with optimized architecture and other hyper-parameters in predicting the COP. Motivated by this, in this paper, five different DL models such as LSTM, Bidirectional LSTM (BiLSTM), GRU, CNN, and convolutional LSTM (ConvLSTM) are considered and optimized to predict the COP. Although probabilistic forecasting of COP is important to tackle the uncertainties occurring due to macroeconomic factors, geopolitical events, supply, and demand dynamics, choice of a forecasting model, and its associated parameters, most of the studies employing DL models have computed only deterministic forecasts for COP [5–15,18]. Although [16,17] have computed the probabilistic forecasting of COP, the architecture and other hyper-parameters of the employed DL models are chosen in an ad-hoc manner which questions the reliability of drawn conclusions. Therefore, in this paper, probabilistic forecasts are computed in addition to deterministic forecasts.

The literature revealed the use of individual models [6–8], ensemble methods [12,17], series hybrid methods [10,11,20], and decomposition-based hybrid methods [9,13–16,18,19] employing DL models to forecast the COP. Despite the application of statistical, ML, DL, and hybrid models for COP forecasting, the full potential of neither individual nor hybrid models in predicting COP has been explored. This is due to the ad-hoc selection of the model and its associated parameters employed in the forecasting model. For example, (i) a statistical model like ARIMA will provide better forecasts if the model order *p, d, q* and its associated parameters are optimized for COP, (ii) a DL model will provide better forecasts if it's architectural parameters (number of layers and number of neurons in each layer) and other hyper-parameters like activation function, kernel initializer, batch size, learning rate, dropout rate, etc. are optimized for COP, (iii) the hybrid models will provide better forecasts if the choice of constituent models are better and the constituent models are optimized for COP. Motivated by this, in this paper, hybrid COP forecasting methods are developed using optimized statistical and optimized DL models.

The architectural and other hyper-parameters of a DL model significantly affect the performance of the underlying method [21–37]. Despite this fact, in the studied literature except [13,20], none of the studies have tuned the architectural and other hyper-parameters of the employed DL models while forecasting the COP data. In [20], the authors have used grid search to tune the hyper-parameters of the LSTM model. They have considered only batch size, epochs, optimizer, dropout rate, and number of neurons in the single hidden layer as the hyper-parameters to tune. However, the other key hyper-parameters, such as the number of hidden layers, activation function, kernel initializer, and learning rate, are ignored. Furthermore, [20] uses the grid search where only the predefined list of values is used as the hyper-parameters of the DL model and is more appropriate if the hyper-parameter has finite discrete values (as in hyper-parameters like activation function, kernel initializer, optimizers, etc.). The grid search is inappropriate to handle continuous valued hyper-parameters like learning rate, dropout rate, etc. Additionally, grid search suffers from dimensionality issues when the number of hyper-parameters to tune becomes large. On the other hand, random search methods like the swarm and evolutionary algorithms are quite efficient in handling continuous and discrete-valued variables. For this, Jiang et al. [13] proposed a decomposition ensemble-based DL approach for deterministic forecasting of COP. The authors have used the Seagull *meta*-heuristic optimization algorithm to optimize the number of neurons in the single hidden layer, optimizer, batch size, and epoch hyper-parameters of the employed GRU model. However, they have ignored the other key hyper-parameters like the number of layers, activation function, kernel initializer, and learning rate. Additionally, they have considered only discrete-valued hyper-parameters of the GRU model and considered only one hidden layer with fixed-length encoding which is not flexible enough to encode DL models with different numbers of layers. As some of the hyper-parameters of DL models are continuous-valued (learning rate and dropout rate) while some are discrete-valued (number of layers, number of neurons in each layer, activation function, kernel initializer, batch size), in this paper we have proposed a novel DE-DL method employing one of the most popular evolutionary algorithms the differential evolution (DE) algorithm to obtain the near-optimal architecture (number of layers and number of neurons in each layer) and other hyper-parameters (activation function, kernel initializer, dropout rate, learning rate and batch size) of a DL model. In the proposed DE-DL method, a novel encoding scheme is proposed that has the simplicity of a fixed-length encoding scheme and the flexibility of a variable-length encoding scheme to encode DL models with different numbers of layers. The proposed DE-DL method is applied to tune the hyper-parameters of five popular DL models: LSTM, CNN, BiLSTM, ConvLSTM, and GRU. The optimized DL models are used for deterministic and probabilistic forecasting of COP. Additionally, optimized additive and multiplicative series hybrid methods are proposed employing optimized DL and optimized ARIMA models for COP forecasting.

To differentiate between the present study and recent literature related to COP forecasting using DL models, the COP data, DL model, consideration of an optimized architecture of DL model, type of forecast, and method employed in the reviewed recent literature and present paper is shown in Table 1.

Although optimized DL models (with optimized architecture and other hyper-parameters) have not been applied in COP forecasting, the architecture and other hyper-parameters of DL models are optimized and used in other application domains like energy forecasting [21], water quality prediction [22], image classification [27,28,30–34,37–39], time series forecasting [44], and wind speed prediction [24,35]. Different DL models, including LSTM [21,25,36], BiLSTM [22,23,35], CNN [26–28,30–34,36–39,44–48], RNN [29,36], GRU [21,33], deep belief network (DBN) [24] have been optimized for different applications. However, the optimization of CNN dominated the recent literature. Wei et. al. [40] presented a leader population learning rate schedule (LPLRS) algorithm that can be seamlessly integrated with any current learning rate schedule and optimizer to leverage the population feature of distributed training and enhance the model's overall performance. However, more sophisticated population-based algorithms incorporating more hyperparameters for joint optimization have been proposed. In another study, Rao and Meher [41] presented a three-stage classification framework employing an optimized ResNet, GoogleNet, and Radial Basis Function-GRU (ORG-RGRU) to analyze speech signals for diagnosing Parkinson's disease. The empirical wavelet transform (EWT) was used to decompose the signals and the African vultures optimization algorithm (ACP-AVOA) was used to tune the hyperparameters of ResNet, GoogleNet, and ORG-RGRU. Viadinugroho and Rosadi [42] presented a multi-objective optimization algorithm by using a weighted metric scalarization method and Bayesian Optimization-Hyperband (BOHB) in the LSTM model for sentiment analysis. The model training time and the F1-score (Macro) were considered as the objective functions. In another study, Meng et al. [43] developed a self-organizing fuzzy neural network (FNN) with a hybrid learning algorithm (SOFNN-HLA) for modeling wastewater treatment processes. Further, in [44], Hong and Chen proposed an OPSO-CNN method, which integrates an opposition-based particle swarm optimization (OPSO) to optimize the hyper-parameters of CNN for semiconductor photomask defect classification. The hyper-parameters of DL models commonly used for optimization include number of neurons in each layer [29,31,32,35,36,42], activation function [23,26,29,30,33,34], learning rate [21,24,25,27,28,30,33,35,38,39,40,41,42,44], batch size [23,26–28,30,33,38,44], dropout rate [21,22,25,27,30,33,34,42], and kernel initializer (weight initialization scheme) [26,32,34,36]. However, all these hyper-parameters are simultaneously not optimized in any of the existing studies. Therefore, in the proposed DE-DL method, we have considered these key hyper-parameters of DL models to tune and developed the method in such a way that it can be applicable to different DL models for any supervised learning problem. In the literature, *meta*-heuristic algorithms such as evolutionary algorithms (EA) [29,37–39], PSO [22,24,31,44], DE [32], genetic algorithm (GA) [23,28,33,34,36], fractal decomposition algorithm (FDA) [27], cuckoo search optimization (CSO) [30], sine cosine algorithm (ISCA) [21,35], Artificial Rabbits Optimization Algorithm (ARO) [25], Honey Badger Algorithm (HBA) [26] have been applied to optimize the DL models. Looking into the efficiency, fewer algorithmic parameters, and chances to avoid the local optima, we have used the most promising DE algorithm in our proposed DE-DL method. Even if, the DE algorithm [32] is used for optimizing CNN models with variable-length real encoding schemes for image classification, it is not a generic one to be applied to different DL

Table 1
Literature on Crude Oil Price Forecasting using Deep Learning Models.

Reference	Type of Crude Oil Data	DL Model	Optimized Architecture	Type of Forecast	Method
[6]	West texas intermediate (WTI), Brent (Daily closing price)	LSTM	No	Deterministic	Individual
[7]	INE Crude oil future prices (Monthly)	LSTM, RNN, GRU	No	Deterministic	Individual
[8]	WTI (Daily closing price)	MultivariateLSTM	No	Deterministic	Individual
[9]	News headlines and Brent crude oil futures prices (Daily)	BiLSTM	No	Deterministic	Decomposition based Hybrid
[10]	WTI (Daily closing price)	GRU, LSTM	No	Deterministic	Series Hybrid
[11]	WTI (Monthly), Online news related to crude oil.	RNN, LSTM	No	Deterministic	Series Hybrid
[12]	Crude oil with predefined prices from Bloomberg (Monthly)	DRNN	No	Deterministic	Ensemble
[13]	News headlines & WTI crude oil price (Daily)	GRU	Yes (Seagull Optimization algorithm)	Deterministic	Decomposition based Hybrid
[14]	Brent (Weekly and Daily closing prices)	LSTM	No	Deterministic	Decomposition based Hybrid
[15]	WTI (Daily closing prices)	GRU	No	Deterministic	Decomposition based Hybrid
[16]	WTI, Brent (Daily closing prices)	GRU	No	Deterministic	Decomposition based Hybrid
[17]	Brent (Daily closing prices)	LSTM, BiLSTM	No	Probabilistic	Ensemble
[18]	WTI, Brent, OPEC (Daily closing prices)	CNN	No	Probabilistic	Decomposition based Hybrid
[19]	WTI (Daily and monthly closing prices)	LSTM	No	Deterministic	Decomposition based Hybrid
[20]	Daily closing prices	LSTM	Yes (Grid Search)	Deterministic	Series Hybrid
Proposed	WTI (Monthly)	LSTM, CNN, GRU, BiLSTM, ConvLSTM	Yes (Differential Evolution)	Deterministic and Probabilistic	Individual and Series Hybrid

models for various applications. In contrast, the proposed DE-DL method is simple, efficient, and so generic that it can be applied to optimize different DL models and solve different supervised problems. In the literature, three encoding schemes are used to encode the DL models: real-valued, block, and binary encoding. Each encoding scheme may be fixed-length or variable-length. The DL models are encoded using fixed-length binary [31], variable-length binary [39], fixed-length real [21–24,26–29,35,37,38], variable-length real [25,30,32,36,38], fixed-length block [33] or variable-length block [34] encoding scheme. Out of six encoding types, fixed-length real

Table 2
Literature on Optimized Deep Learning Models for Different Applications.

Reference	Deep Learning Model	Hyper-Parameters	Optimization Technique	Encoding Scheme	Application
[21]	LSTM, GRU	Number of neurons in the first hidden layer, learning rate, number of training epochs, dropout rate, and number of neurons in the second hidden layer.	Boosted Self-Adaptive SCA	Fixed-length real encoding	Energy Forecasting
[22]	Bi-LSTM	Number of layers, dropout rate, sequence length.	Genetic Simulated annealing-based PSO	Fixed-length real encoding	Water Quality Prediction
[23]	Bi-LSTM	Batch size, sampling interval, optimizer, activation function, and number of neurons in each hidden layer.	GA	Fixed-length real encoding	Water Industry
[24]	DBN	Number of hidden layers, number of neurons in each hidden layer, and learning rate.	Multi-objective sine cosine PSO	Fixed-length real encoding	Wind Speed Prediction
[25]	LSTM	Number of neurons, existence of layer, dropout rate, optimizer, learning rate.	ARO	Variable-length real Encoding	Stock price prediction
[26]	CNN	Number of filters, size of kernel, activation function, pool size, optimizer, batch size.	HBA	Fixed-length real encoding	Text Classification
[27]	CNN	Dropout rate, number of neurons in the dense layers, momentum, batch size, number of epochs, number of filters, learning rate, weight decay.	FDA	Fixed-length real encoding	Image Classification
[28]	CNN	Learning rate, channels, weight decay, momentum, optimizer, batch size, number of epochs.	GA	Fixed-length real encoding	Image Classification
[29]	RNN	Activation function, net input function, learning methods, model of neuron, and number of neurons in the output layer.	EA + Gradient Descent Method	Fixed-length real encoding	Nuclear Reactor
[30]	CNN	Batch size, number of epochs, optimizer, momentum rate, learning rate, dropout rate, activation function, number of convolutional layers, number of filters, convolution filter size, and max-pooling size.	CSO	Variable-length real encoding	Image Classification
[31]	CNN	Number of neurons in each convolutional layer, and number of convolutional layers.	PSO	Fixed-length binary encoding	Image Classification
[32]	CNN	Number of neurons in a Fully Connected layer, pool type, pool stride size, pool kernel size, convolution kernel size, convolutional stride size, length of CNN, and number of feature maps.	DE	Variable-length real encoding	Image Classification
[33]	CNN	Number of blocks, number of nodes, number of filters, filter size, pooling, dropout, short connection, batch normalization, type of layer, activation function, dropout, optimizer, learning rate, batch size, initialization, and number of convolution layers.	GA	Fixed-length Block-Based encoding	Image Classification
[34]	CNN	Dropout rate, activation function, kernel initializer, position of batch normalization, batch normalization control parameter, number of hidden neurons, and number of convolutional kernels.	GA	Variable-length Block-Based encoding	Image Classification
[35]	BiLSTM	Number of epochs, learning rate, number of iterations, number of hidden neurons.	ISCA	Fixed-length real encoding	Wind Speed Prediction
[36]	CNN,RNN,GRU, LSTM	Network type, weight value, number of CNN layers, kernel size of CNN, whether bidirectional of the recurrent neural network, number of layers of the recurrent neural network, and number of hidden nodes of the recurrent neural network.	GA	Variable-length real encoding	Time Series Forecasting
[37]	CNN	Learning method, momentum rate, weight decay, drop path probability.	EA	Fixed-length real encoding	Image Classification
[38]	CNN	Batch size, epochs, learning rate, and momentum.	EA	Fixed-length binary encoding Variable-length real encoding	Image Classification
[39]	CNN	Learning rate for individual evaluation and fine-tuning, number of epochs for individual evaluation and fine-tuning.	EA	Variable-length binary encoding	Image Classification
This Paper	LSTM, CNN, GRU, BiLSTM, ConvLSTM	Number of layers, number of neurons in each layer, number of filters (in case of CNN and ConvLSTM), number of LSTM units (in case of LSTM, BiLSTM, ConvLSTM), activation function, kernel initializer, dropout rate, batch size, learning rate.	DE	Proposed fixed-length real encoding	Crude Oil Price Forecasting

encoding is easier to implement and computationally cheaper [27]. At the same time, the variable-length encoding scheme is flexible to encode DL models with varying numbers of layers [30]. However, the fixed-length real encoding schemes found in the literature are not flexible enough to encode DL models with varying numbers of layers. Therefore, in the proposed DE-DL method, we have used a novel fixed-length encoding scheme with a pruning method that combines the simplicity of a fixed-length real encoding scheme with the flexibility of a variable-length encoding scheme to encode DL models with different numbers of layers. To compare the difference between the proposed DE-DL method and other DL optimization methods existing in the literature, the considered DL models, considered hyper-parameters for tuning, optimization techniques, encoding schemes, and application domains employed in the reviewed recent literature and the present paper is presented in Table 2.

The contributions of the paper are as follows:

(i) A DE-DL method employing the DE algorithm is proposed to optimize the architecture and other hyper-parameters of a DL model for a supervised problem.
(ii) The architecture and other hyper-parameters of five DL models (LSTM, BiLSTM, CNN, GRU, and ConvLSTM) are optimized for forecasting the monthly COP using the proposed DE-DL method.
(iii) Six statistical models are optimized for forecasting monthly COP using the Forecast Package of R.
(iv) Five optimized additive hybrid models (aARIMA-LSTM, aARIMA-BiLSTM, aARIMA-CNN, aARIMA-GRU, and aARIMA-ConvLSTM) employing the optimized DL models and optimized ARIMA model are developed for forecasting the monthly COP.
(v) Five optimized multiplicative hybrid models (mARIMA-LSTM, mARIMA-BiLSTM, mARIMA-CNN, mARIMA-GRU, and mARIMA-ConvLSTM) employing the optimized DL models and optimized ARIMA model are developed for forecasting the monthly COP.
(vi) A recurrence plot is drawn, recurrence quantification analysis (RQA), and uncertainty analysis of prediction error are performed.
(vii) Probabilistic COP forecasts are computed with 90 %, 80 %, 70 %, and 60 % confidence levels by modeling the errors from the best deterministic forecasts and using Gaussian and t-Location Scale distributions.
(viii) Extensive non-parametric statistical analysis of the obtained results are carried out to rank the five optimized DL models, six optimized statistical model, thirteen machine learning models, and ten additive and multiplicative hybrid models for deterministic and probabilistic forecasting of monthly COP using four deterministic performance measures and four probabilistic performance measures.

In addition, the following eight research questions related to deterministic and probabilistic forecasting of COP employing optimized statistical, DL, and hybrid models are addressed:

RQ-1: Which optimized DL model among LSTM, CNN, GRU, BiLSTM, and ConvLSTM provides statistically better results for predicting the monthly COP?

This question helps to investigate the performance of different DL models (with optimal architecture and hyper-parameters) in predicting the monthly COP.

RQ-2: Which optimized statistical model is the best model for predicting monthly COP?

This question helps to investigate the performance of different statistical models (with optimal model parameters) in predicting the monthly COP. It will also help in identifying the statistical model to be used in additive and multiplicative hybrid models.

RQ-3: Which optimized additive hybrid method employing optimized statistical and DL model provides statistically better results for predicting the monthly COP?

This question helps to investigate the performance of different additive hybrid methods employing optimized DL models and optimized statistical models for predicting the monthly COP.

RQ-4: Which optimized multiplicative hybrid method employing optimized statistical and DL model provides statistically better results for predicting the monthly COP?

This question helps to investigate the performance of different multiplicative hybrid methods employing optimized statistical and DL models for predicting the monthly COP.

RQ-5: What is the ranking of optimized DL, optimized statistical, additive and multiplicative hybrid models in predicting the monthly COP?

This question helps to identify the best model (when optimized DL and statistical models are used) to predict the monthly COP.

RQ-6: Which distribution function is the right alternative for probabilistic forecasting of monthly COP?

This question helps in determining the right alternative in modeling the errors using distribution functions for computing the prediction intervals.

RQ-7: Are the probabilistic predictions obtained using optimized statistical and DL models reliable and with desired resolution?

This question helps in determining the usefulness and reliability of prediction intervals obtained using the best method employing optimized statistical and DL models.

RQ-8: What is the computational complexity of the best method obtained from this study?

This question helps in determining time and cost estimation, hardware and resource allocation, efficiency, and scalability of the best COP forecasting method employing optimized DL and statistical models.

The remaining of this paper is organized as follows. Section 2 presents the proposed DE-DL method for architecture and hyper-parameter optimization of DL models. Section 3 presents the methodology employed to predict the COP using optimized statistical, DL, and hybrid methods. Section 4 presents the simulation results for deterministic and probabilistic forecasting of COP employing

optimized statistical, DL, and hybrid models. Section 5 discusses the results. Section 6 concludes the paper.

2. Proposed DE-DL method for optimization of architecture and other hyper-parameters of DL models

Optimizing the DL models means obtaining the right architecture and hyper-parameters for a problem from the search space of all combinations of architectures and other hyper-parameters. It can be considered as a constrained stochastic search problem from the space of all possible architectures and hyper-parameters. In this paper, we have chosen the DE algorithm as the stochastic search algorithm because of its simplicity, efficiency, and ability to search in a search space having both continuous-valued and discrete-valued variables. As the DE algorithm is used to optimize the DL model parameters, we have given the name DE-DL for the employed method.

To apply DE for optimizing the DL model hyper-parameters, first, the promising hyper-parameters need to be identified. The performance of DL models largely depends on the number of layers, the number of neurons in each layer, the initial value of weight sets (kernel initializer), activation functions, dropout rate (to avoid overfitting), learning rate (too small value leads to slow convergence and too large value leads to oscillation) and batch size. Therefore, we have chosen four hyper-parameters for each hidden layer which are the number of neurons (filters, LSTM units or GRU units), activation function, kernel initializer (initial weight set initialization scheme) and dropout rate. Additionally, two other hyper-parameters of the DL model, i.e. batch size and learning rate are considered. The number of layers is also determined based on a pruning method where initially, a larger DL model is encoded, and based on the dropout rate, the layers are pruned. Out of all these hyper-parameters, the activation function and kernel initializer are considered as discrete-valued hyper-parameters, and others are considered as continuous-valued. We have used four alternatives in the activation function (relu, sigmoid, tanh, elu) and four alternatives in the kernel initializer (glorot_uniform, glorot_normal, he_uniform, he_normal). Therefore, the lower limit is set to zero, and the upper limit is set to three for the decision variables denoting the activation function and kernel initializer. The decoding algorithms for the activation function and kernel initializer are presented in Algorithm 1 and Algorithm 2 respectively. The upper and lower limit of the number of neurons in a layer is set to 2 and 256, respectively. While decoding a decision variable v for the number of neurons, the rounded-off value for v is used. The upper and lower limit of the dropout rate is set to 0 and 1, respectively. Our method has dropped a layer when the dropout rate is 1. The batch size is usually taken as a power of 2. The lower limit and upper limit are set to 0 and 7, respectively. While decoding a decision variable v to batch size, the rounded-off value of v raised to the power of 2 is used as the batch size. Hence, the batch size varies between 1 and 128, which is a power of 2. The lower and upper limits of learning rate are set to 0.0001 and 0.1, respectively.

Algorithm 1
Get the activation function: get_activation_function(v).

Input: Decision variable v
Output: Activation function
1: gene ← round (v)
2: **if** gene equals to 0 **then**
3: **return** "relu"
4: **else if** gene equals to 1 **then**
5: **return** "sigmoid"
6: **else if** gene equals to 2 **then**
7: **return** "tanh"
8: **else**
9: **return** "elu"
10: **endif**

Algorithm 2
Get the kernel initializer: get_kernel_initializer(v).

Input: Decision variable v
Output: Kernel Initializer
1: gene ← round (v)
2: **if** gene equals to 0 **then**
3: **return** "glorot_uniform"
4: **else if** gene equals to 1 **then**
5: **return** "glorot_normal"
6: **else if** gene equals to 2 **then**
7: **return** "he_uniform"
8: **else**
9: **return** "he_normal"
10: **endif**

Algorithm 3 presents the decoding of a decision vector to a DL model. Real encoding (as in Table 1 for a 2-hidden layer DL model) is used to encode a DL model in a decision vector of the DE algorithm. In a decision vector, first, the four hyper-parameters of each hidden layer are kept one after another and then the learning rate and batch size of the DL model are kept (as in Fig. 1). The inputs, the number and type of output layer neuron(s) are predetermined and is based on the supervised problem in hand. In the monthly COP forecasting problem, we have kept the number of inputs to 12 and number of neurons in the output layer to one with a linear activation function. For example, consider a DL model with a maximum of two hidden layers. Then the encoded decision vector can be as shown in Table 3.

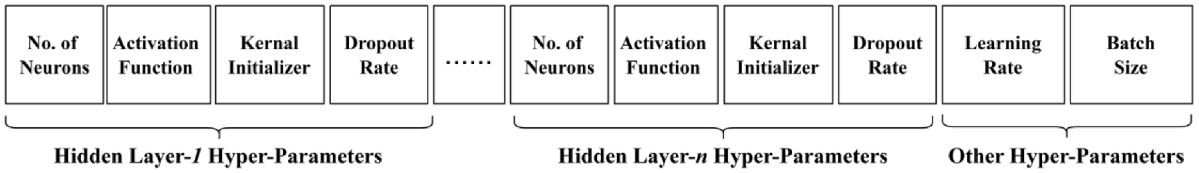

Fig. 1. Encoding a DL model to a decision vector (chromosome).

Table 3
Example of real encoded DL model with hyper-parameters.

128.33	1.2	2.4	0.7	69.78	2.6	1.3	0.4	0.01	3.89

Using Algorithm 1, Algorithm 2 and Algorithm 3, the first hidden layer of decoded DL model has 128 neurons with a "sigmoid" activation function, "he_uniform" as the kernel initializer, and has a dropout rate of 0.7. The second hidden layer has 70 neurons with "elu" activation, and "glorot_normal" as kernel initializer and has a dropout rate of 0.4. The learning rate of the network is 0.01 and the batch size is 16. If the dropout rate of the second hidden layer is one (as in Table 4), then that layer is dropped and the DL model has only one hidden layer. Similarly, if a DL model has a maximum of k-hidden layers, then using the proposed encoding scheme, the DL model may have 0 to k-hidden layers. Hence, the number of layers is also tuned in the proposed DE-DL method.

Algorithm 3
Decode a Decision Vector to a DL Model.

Input: Decision vector $V = [v_1, v_2, \cdots, v_d]$
Output: DL Model
1: model DL ← empty
2: add layer input to DL
3: **for** each hidden layer hyper-parameter in decision vector V
4: **if** v_{i+4} not equal to 1 **then** // dropout!=1
5: nn = round(v_i) // number of neuron
6: af = get_activation_function(v_{i+1})
7: ki = get_kernel_initializer(v_{i+2})
8: **add** a layer to DL model with nn neurons, af as activation function, ki as kernel initializer and v_{i+4} as dropout rate.
9: **endif**
10: **endfor**
11: **add** a layer with 1 neuron and linear activation function. // Output layer
12: batch_size=$2^{round(v_d)}$
13: learning_rate=v_{d-1}
14: Compile the DL model with mean square error (for a regression problem) or multiclass logloss (for a classification problem) as the loss function, Adam as the optimizer with learning rate set to learning_rate.
15: **return** DL

Algorithm 4
Steps of the Proposed DE-DL method for optimization of DL Models.

Input: Maximum number of hidden layers (k), Input (X), and Target (y).
Output: Optimized DL model with near-optimal hyper-parameters.
1: Initialize the DE algorithmic parameters such as population size, scale factor (F), crossover probability (Cr), number of decision variables in each decision vector (dim), maximum generations (max_gen), and upper and lower bound of decision variables. Split the input and target patterns into train, validation and test sets.
2: Randomly initialize the population size number of decision vectors from a uniform distribution within the upper and lower bound of the decision variables. Each decision variable of a decision vector denotes a hyper-parameter of the DL model. Each decision vector denotes a DL model with associated hyper-parameters. The number of decision variables in a decision vector (dim) is set to $k \times 4 + 2$ i.e., 4 hyper-parameters(number of neurons, activation function, kernel initializer, and dropout rate) for each hidden layer and 2 additional hyper-parameters (learning rate and batch size) for the DL model.
3: Decode each decision vector (using Algorithm 3) to get the DL model and calculate its fitness using the Adam algorithm on the train and validation set.
4: **While** (termination criteria like reaching max_gen are not satisfied) do begin
5: **for** each decision vector (called target vector) in the population
6: Compute the mutant vector using the DE/best/1 mutation scheme.
7: Compute the trial vector by performing binomial crossover between the mutant vector and target vector.
8: Perform selection between the trial vector and target vector based on fitness which is obtained by using the Adam algorithm on train and validation set. The better vector between the trial and target vector moves to the next generation.
9: **endfor**
10: **endwhile**
11: Decode the decision vector of the final generation with the best fitness using Algorithm 3 to obtain the optimized DL model with associated hyper-parameters. This optimized DL model is used to compute the predictions on the test set and performance is measured.

Table 4
Example of real encoded DL model with hyper-parameters and pruning of layer.

128.33	1.2	2.4	0.7	69.78	2.6	1.3	1	0.01	3.89

Algorithm 4 presents the steps of the proposed DE-DL method employing the DE algorithm to determine the architectural and other hyper-parameters of a DL model for a supervised problem. It takes the maximum number of hidden layers of the DL model (k), input (X), and target (y) as the input and produces the near-optimal architecture and other hyper-parameters of the DL model with its performance on the test set as output. It operates in three steps Initialization, Iteration, and Final step. In the initialization step, the DE

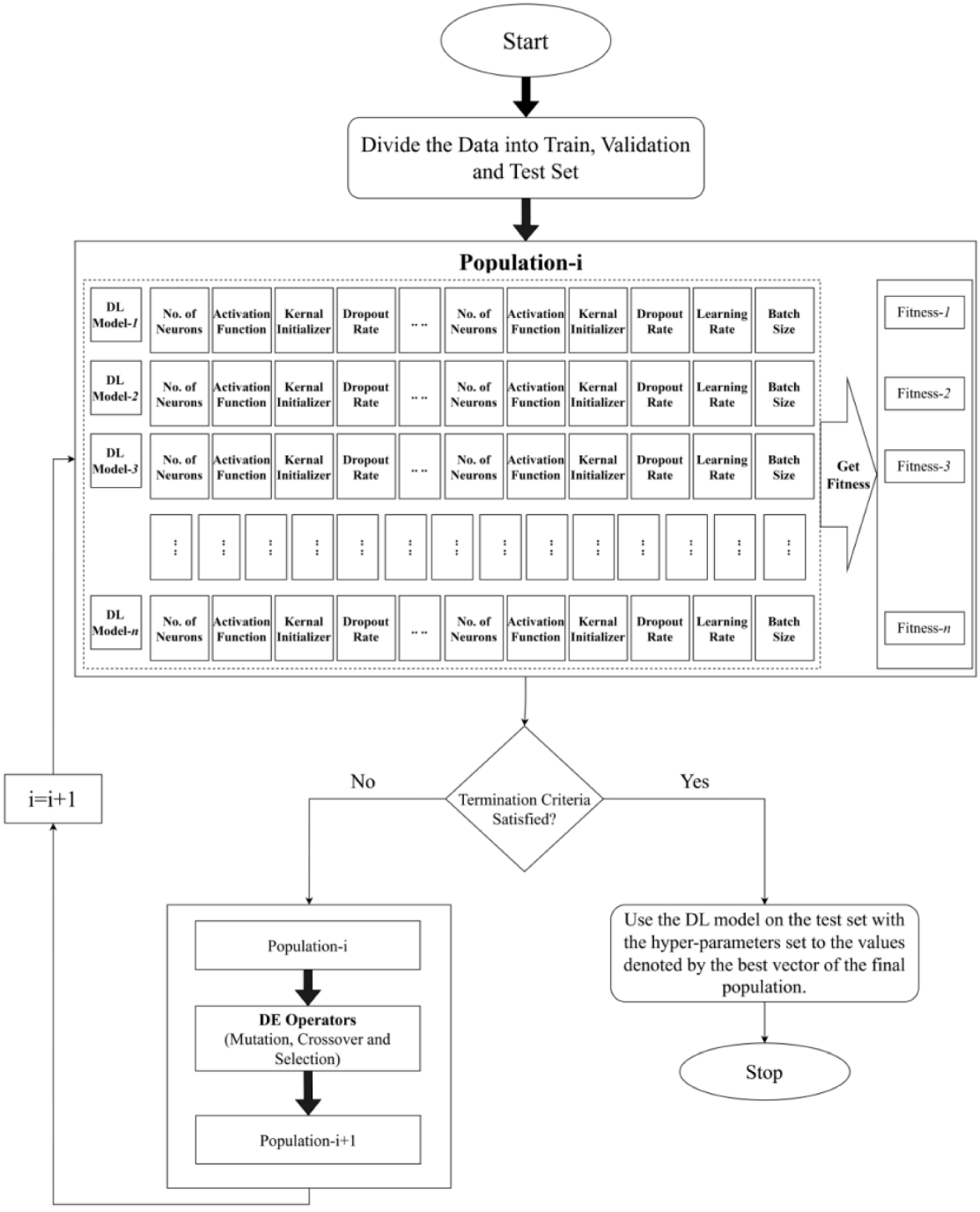

Fig. 2. Graphical abstract of proposed DE-DL method.

algorithmic parameters like population size (P_{size}), scale factor (F), crossover probability (Cr), number of decision variables in each decision vector, and upper and lower bound of decision variables are initialized. The input and target patterns are split into train, validation, and test sets. Then, the initial population is randomly generated i.e., P_{size} number of decision vectors are randomly generated from a uniform distribution within the upper and lower bounds of the corresponding decision variables. Each decision variable of a decision vector denotes a hyper-parameter of the DL model. Each decision vector denotes a DL model with associated hyper-parameters. The number of decision variables in a decision vector is set to $k \times 4 + 2$ i.e., 4 hyper-parameters (number of neurons, activation function, kernel initializer, and dropout rate) for each hidden layer and 2 additional hyper-parameters (learning rate and batch size) for the DL model. Then, each decision vector is decoded using Algorithm 3 to get the DL model and its fitness is calculated by using the Adam algorithm on the train and validation set. The DL model is trained using the train set, and their fitness is evaluated based on the validation set. The objective of the fitness function (as in Eq. (1) is to minimize the mean square error by using the decoded DL model and its associated hyper-parameters (as in ith decision vector $\boldsymbol{V}_i$). Once the initial population is generated and the fitness of each decision vector is evaluated, the iteration step begins. In the iteration step, mutation, crossover, and selection operators of the DE algorithm are repeatedly applied to generate the next generations. For this, at first, the DE mutation operator is applied to generate the mutant vector. In simulations, we have used the DE/best/1 mutation scheme for its simplicity and faster convergence. To apply DE/best/1 mutation, the best decision vector of the current generation and two randomly selected decision vectors from the current generation are selected in such a way that the best vector, two randomly selected vectors, and the target vector are distinct from one another. Then the difference vector between the two randomly selected vectors is computed which is multiplied by the scale factor and then added with the best vector to obtain the mutant vector. In this paper, the scale factor (F) is randomly generated from a uniform distribution within the range 0 to 1 because when the difference vector is small (i.e. two close vectors are selected) then larger values of F will be used to get out of the local optima and suboptimal valleys. When the difference vector is large (i.e. when two vectors are far away from each other), then a smaller scale factor F will assist in searching around the base vector. Thus, a trade between exploration and exploitation will be maintained, which will assist in reaching the near-optimal solutions in fewer iterations. The mutant vector is then checked for boundary value violation. If the value of any decision variable of the mutant vector is more than its upper bound then its value is set to the corresponding upper bound and if it is less than the lower bound then it is set to the value of the corresponding lower bound. Once the mutant vector is generated, it is crossover with the target vector using DE binomial crossover to generate the trial vector. The binomial crossover combines the decision variables from the mutant vector and target vector to generate a trial vector. In the binomial crossover, the decision variables of the trial vector keep the value of the corresponding mutant vector's decision variable if a randomly generated number is less than the crossover probability (Cr) otherwise it keeps the value of the target vector's corresponding decision variable. In the simulations, the crossover probability is set to 0.7. Then selection operator is applied between the trial vector and target vector to move the better fitness decision vector to the next generation. The next generation of decision vectors is repeatedly generated until a termination criterion, like reaching a maximum number of generations is satisfied. Once the termination criterion is reached, in the final step, the best vector of the final population denotes the DL model with the best architecture and associated hyper-parameters which is used to predict the test set, and the performance is measured. For a better understanding, the graphical abstract of the proposed DE-DL method is presented in Fig. 2.

$$\min_{V_i} inf(V_i) \tag{1}$$

Subjected to $L_j \leq V_{i,j} \leq U_j$

where $f(\bullet)$ denotes the mean square error using the DL Model with hyper-parameters encoded in V_i, L_j and U_j denotes the lower limit and upper limit of the jth hyper-parameter.

The DE-DL method is applied to optimize five DL models (LSTM, CNN, GRU, BiLSTM, ConvLSTM) for monthly COP forecasting by setting the maximum number of hidden layers to 2. The population size is fixed to 50, and the maximum number of generations is set to 30. DE/best/1/bin scheme is used with two control parameters, scale factor $F = \text{rand}(0,1)$ and crossover probability $Cr = 0.7$. The obtained best DL model hyper-parameters for LSTM, CNN, GRU, BiLSTM, and ConvLSTM are presented in Table 5. It can be observed

Table 5
Obtained optimized DL models with their Hyper-Parameters for monthly COP data using the proposed DE-DL method (Italics face denotes hyper-parameter values of a dropped layer).

	Hyper-Parameter	LSTM	CNN	GRU	BiLSTM	ConvLSTM
Hidden Layer-1	Number of Neurons	2	216	198	80	106
Hyper-parameters	Activation Function	Elu	Elu	Elu	tanh	Elu
	Kernel Initializer	glorot_uniform	he_normal	glorot_uniform	glorot_normal	glorot_normal
	Dropout Rate	0	0	0.18	0	0.19
Hidden Layer-2	Number of Neurons	*256*	*158*	192	204	256
Hyper-parameters	Activation Function	*Sigmoid*	*Sigmoid*	Tanh	Elu	elu
	Kernel Initializer	*glorot_uniform*	*glorot_normal*	he_normal	he_uniform	he_normal
	Dropout Rate	*1*	*1*	0	0.084	0
Other Hyper-Parameters	Learning Rate	0.05	0.0001	0.086	0.009	0.0001
	Batch Size	8	16	16	128	8
Output Layer	Number of Neurons	1	1	1	1	1
	Activation Function	Linear	Linear	Linear	Linear	Linear
Training Algorithm	Optimizer	Adam	Adam	Adam	adam	adam

from Table 5 that the dropout rate for hidden layer-2 of the LSTM and CNN model has a value of 1, which means, as per our proposed method those layers are dropped out of the network, i.e. the LSTM and CNN model has a single hidden layer. However, the GRU, BiLSTM, and ConvLSTM have two hidden layers.

3. Employed methodology to Forecast the COP using optimized Statistical, DL, and hybrid models

In this paper, extensive studies are made to forecast the monthly COP using optimized statistical, DL and resulting hybrid models (additive and multiplicative). Six promising statistical models, namely ARIMA, autoregressive fractionally integrated moving average (ARFIMA), exponential smoothing with error, trend, and seasonality (ETS), TBAT, Theta, and Naïve are optimized using the Forecast package of R [45,46]. The normalized train and validation set of the COP time series is used to optimize the models. Once the optimal statistical model is obtained, it is used to forecast the normalized test data. The normalized forecasts are de-normalized, and forecast accuracy is measured.

Five promising DL models, namely LSTM, CNN, GRU, BiLSTM, and ConvLSTM are optimized using the proposed DE-DL method for forecasting the monthly COP time series. Fig. 3 shows the graphical abstract of the methodology employed to forecast the COP using optimized DL models. First, the number of significant inputs is determined. Since the monthly COP time series is considered, the number of inputs is kept to 12. Then the COP time series is normalized using the Min-Max normalization technique. The normalized time series is transformed to a pattern set using the sliding window technique. The pattern set is divided into the train, validation, and test sets. The optimized architecture and other hyper-parameters of the DL model are obtained by employing the DE-DL method on the train and validation set. Then the normalized forecasts on the test set are computed by using the optimized DL model. The normalized predictions are de-normalized, and the forecast accuracy is measured.

The statistical models assume the COP time series is linear, while the DL models assume the COP time series is nonlinear. However, the COP time series can be assumed to be a combination of a linear component and a nonlinear component which may be additive or multiplicative. Assuming the COP time series as an addition of a linear and nonlinear component, additive hybrid models employing optimized statistical linear models and optimized DL nonlinear models can be developed to maximize the true potential of additive hybrid models (as in Fig. 4). First, the COP time series is modeled using an optimized statistical model (optimized using Forecast Package of R). Then the optimized statistical model forecasts are subtracted from the COP data to obtain the residual series. The residual series is considered nonlinear and modeled using an optimized DL model (optimized using the proposed DE-DL method). Then the final forecasts are obtained by adding the optimized statistical model forecasts with optimized DL model forecasts. Since optimized ARIMA provided the best results among all statistical models, we have used the optimized ARIMA model as the linear statistical model. Also, it is difficult to model the residuals because it might exhibit random oscillations, complex nonlinear patterns, and heteroscedasticity [47]. Therefore, to model the nonlinear component, each of the five optimized DL models was applied independently. Hence, a total of five optimized additive hybrid models are employed in this paper.

Contrasting to the additive hybrid models, the optimized multiplicative hybrid models (as in Fig. 5) assume the COP time series as a multiplication of a linear and a nonlinear component. If the linear and nonlinear components are modeled using optimized linear statistical models and optimized nonlinear DL models, then the true potential of multiplicative hybrid models in predicting the COP

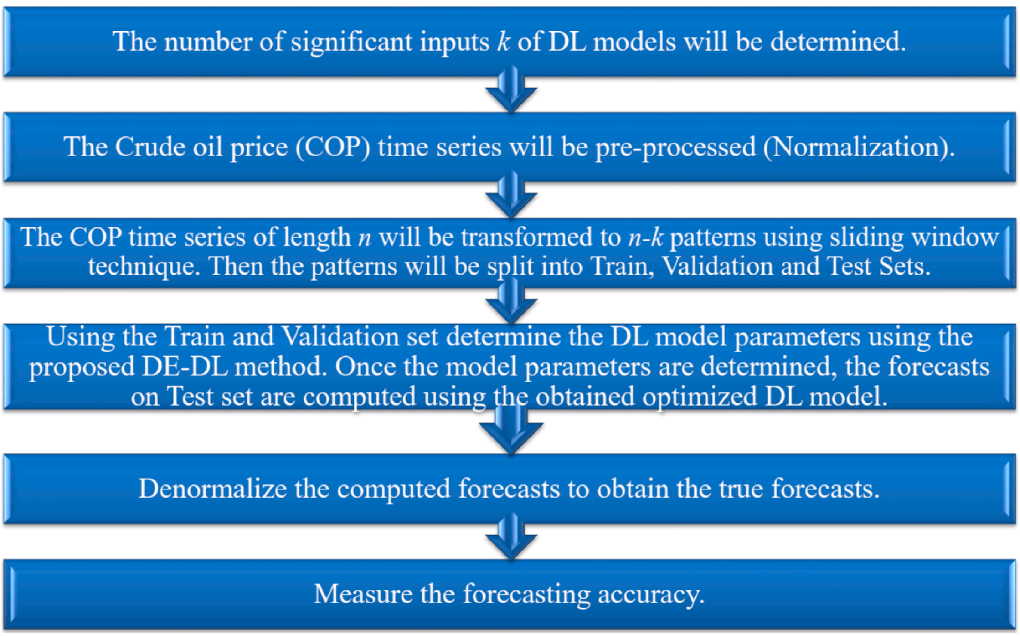

Fig. 3. Graphical Abstract of the methodology employed to Forecast COP using optimized DL Models.

time series can be achieved. For this, first, the COP time series is modeled using an optimized statistical model (optimized using the Forecast package of R). The residual series is obtained by dividing the COP data by the optimized statistical forecasts. Then considering the residual series as nonlinear, it is modeled using an optimized DL model (optimized using the proposed DE-DL method). The final forecasts are obtained by multiplying the optimized statistical model forecasts with the optimized DL model forecasts. Since optimized ARIMA provided the best results among all statistical models, we have used the optimized ARIMA model as the linear statistical model and applied all five optimized DL models separately to model the nonlinear component. Hence, a total of five optimized multiplicative hybrid models are employed in this paper.

4. Simulation results

4.1. Studied crude oil data and recurrence analysis

In this paper, the monthly COP collected from WTI spot price (Dollars per Barrel) between January 1986 and June 2022 is considered for comparative forecasting performance analysis of the models. The COP dataset is split into train (first 60 %, from January 1986 to October 2007, a total of 262 observations), validation (next 20 %, from November 2007 to February 2015, a total of 88 observations), and test set (recent 20 %, March 2015 to June 2022, a total of 88 observations). The train and validation set is considered as in-sample data and used to build the models while the test set is considered as out-sample data which is used to evaluate the models.

To inspect the underlying complex nonlinear patterns of the considered COP, the recurrence plots (RP) and their corresponding recurrence density are shown in Fig. 6. The RP graphically depicts the recurrence behavior of a nonlinear time series. The RP is a square matrix in which each element indicates whether or not the corresponding points in the time series are considered recurrent based on a threshold distance. In colored RPs, the distance between recurrent points is denoted by color. Darker regions represent closer points while lighter regions denote more distant points. The density of recurrence points in an RP indicates the amount of recurrence in the time series. A high density of recurrence points suggests that the system is deterministic and predictable, while a low density of recurrence points suggests that the system is stochastic and unpredictable. Longer and denser diagonal lines denote longer periods for repeated patterns while steeper diagonal lines denote rapid transition between states. A cluster of off-diagonal points denotes periods of recurrence, while sparse points denote more irregular behavior. Vertical lines denote the presence of volatility. It can be observed from the RP (as in Fig. 6) that it has some vertical green lines indicating the presence of weak volatility, has some recurrence points indicating weak periodicity, has neither longer nor steeper diagonal lines, and the RP including the off-diagonal points are neither fully dense nor fully sparse indicating the feasibility of a challenging prediction problem. Furthermore, for a more comprehensive analysis, recurrence quantification analysis (RQA) is carried out [48] to obtain a quantitative measure of the RP. Table 6 presents the RQA results using the PyRQA package of python [48] by setting the time delay to 9 and the embedding dimension to 3.

Table 6 presents the RQA of monthly COP in terms of recurrence rate (RR), determinism (DET), laminarity (LAM), trapping time (TT), and entropy (ENT). The RR specifies the probability that a specific state will recur. A lower value denotes less chance of recurrence of a pattern and hence makes the prediction of the time series challenging. DET specifies the predictability of a dynamic system with zero indicating the absence of a predictable system. The LAM measures the frequency distribution of the length of vertical lines with a minimum length v_{min} which specifies the intermittency in the system. The TT measures the average length of vertical lines

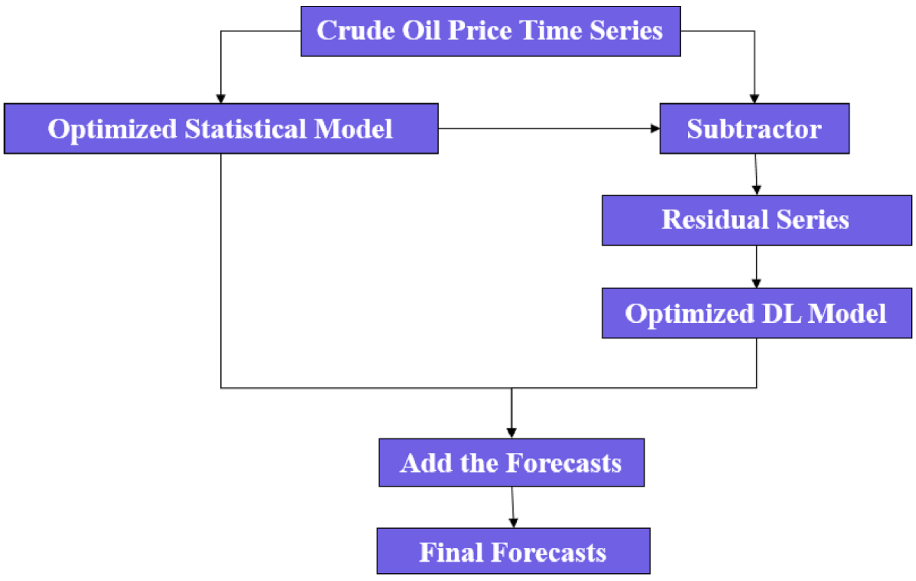

Fig. 4. Graphical abstract of Crude Oil Price Forecasting using Optimized Additive Hybrid Model.

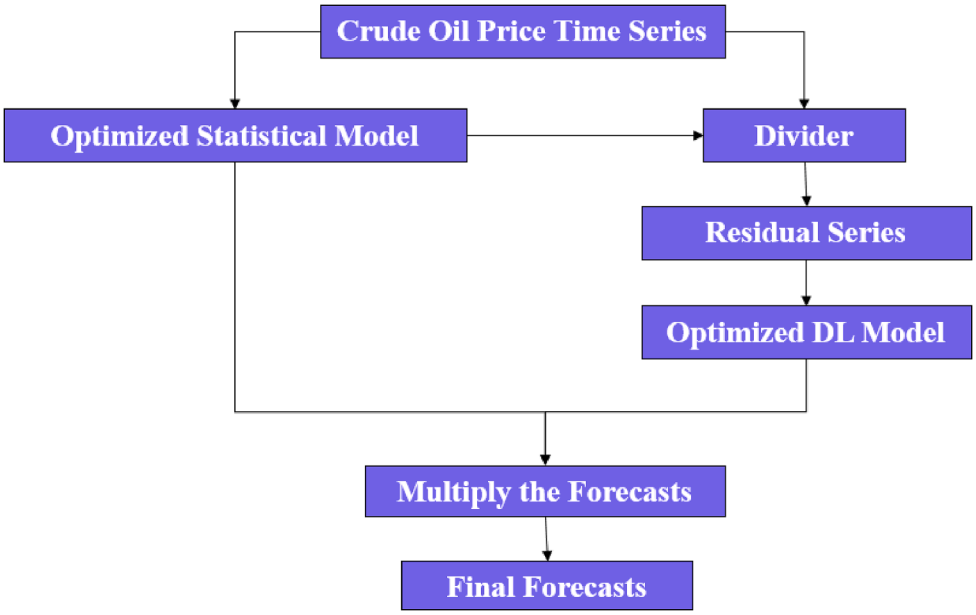

Fig. 5. Graphical abstract of Crude Oil Price Forecasting using Optimized Multiplicative Hybrid Model.

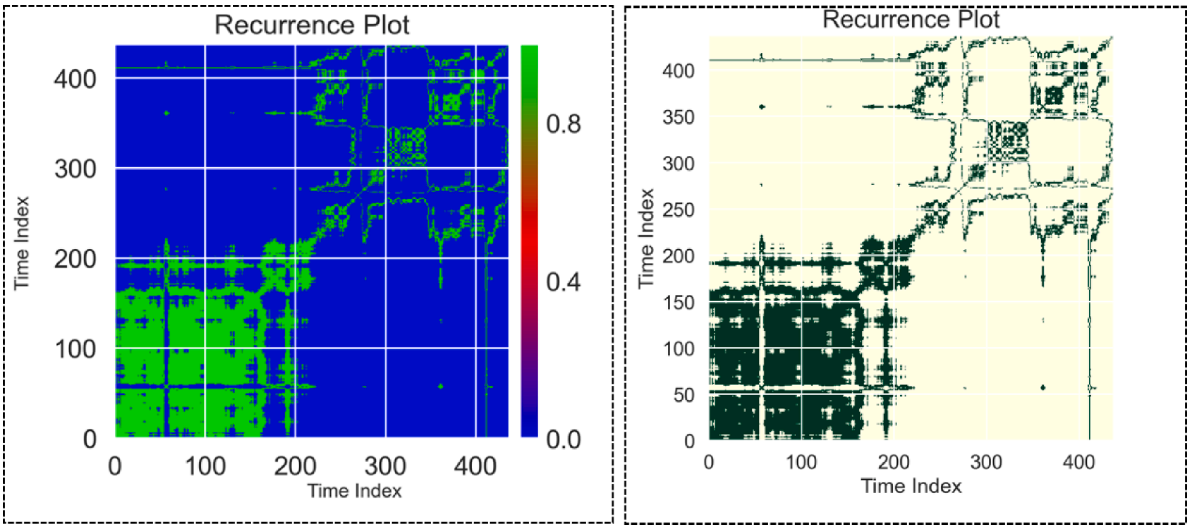

Fig. 6. Recurrence Plots for the Monthly COP.

related to LAM time and specifies how long the system remains in a specific state. ENT specifies the complexity of the deterministic structure in the system. The RQA of monthly COP (as in Table 6) indicates that it can be predictable but is a challenging problem due to a smaller RR with a below-average ENT. Below-average ENT insists on the essence of probabilistic forecasts in addition to deterministic forecasts. Therefore, in this paper, both deterministic and probabilistic forecasting of COP are performed.

Table 6
Recurrence Quantification Analysis of Monthly COP.

RR	DET	LAM	TT	ENT
0.002528	0.153846	0.033632	2.142857	0.410116

4.2. Performance measure

To evaluate the performance of the forecasting methods, we have used four deterministic forecasting accuracy measures, namely root mean square error (RMSE) (Eq. (2), symmetric mean absolute percentage error (SMAPE) (Eq. (3), mean absolute error (MAE) (Eq. (4) and mean absolute scaled error (MASE) (Eq. (5) for evaluating the deterministic forecasts and four measures, namely prediction interval coverage probability (PICP) (Eq. (6), prediction interval normalized average width (PINAW) (Eq. (7), accumulated width deviation (AWD) (Eq. (8), and average coverage error (ACE) (Eq. (9) for evaluating the probabilistic forecasts. The lower values of RMSE, SMAPE, MAE, MASE, PINAW, and AWD are desirable, while the higher values of PICP and ACE are desirable for a better forecasting model.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - y_i^{'})^2} \tag{2}$$

$$SMAPE = \frac{1}{n}\sum_{i=1}^{n}\frac{|y_i - y_i^{'}|}{(|y_i| + |y_i^{'}|)/2} \tag{3}$$

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - y_i^{'}| \tag{4}$$

$$MASE = \frac{1}{n}\sum_{i=1}^{n}\frac{|y_i - y_i^{'}|}{\frac{1}{n-1}\sum_{j=2}^{n}|y_i - y_{i-1}|} \tag{5}$$

$$PICP^{(\alpha)} = \frac{1}{n}\sum_{i=1}^{n}\mathtt{C}_i, \mathtt{C}_i = \begin{cases} 1 y_i \in [L_i^{(\alpha)}, U_i^{(\alpha)}] \\ 0 y_i \notin [L_i^{(\alpha)}, U_i^{(\alpha)}] \end{cases} \tag{6}$$

$$PINAW^{(\alpha)} = \frac{1}{nR}\sum_{i=1}^{n}(U_i^{\alpha} - L_i^{\alpha}) \tag{7}$$

$$AWD^{(\alpha)} = \frac{1}{n}\sum_{i=1}^{n}AWD_i^{\alpha}, AWD_i^{\alpha} = \begin{cases} \frac{L_i^{(\alpha)} - y_i}{U_i^{(\alpha)} - L_i^{(\alpha)}}, y_i < L_i^{(\alpha)} \\ 0, y_i \in [L_i^{(\alpha)}, U_i^{(\alpha)}] \\ \frac{y_i - U_i^{(\alpha)}}{U_i^{(\alpha)} - L_i^{(\alpha)}}, y_i > U_i^{(\alpha)} \end{cases} \tag{8}$$

$$ACE^{(\alpha)} = PICP^{(\alpha)} - PINC^{(\alpha)} \tag{9}$$

where y_i and $y_i^{'}$ denote the ith actual and forecasted COP value, n denotes the number of observations, α denotes the significance level, $L_i^{(\alpha)}$ and $U_i^{(\alpha)}$ denote the lower bound and upper bound for ith data point with α-significance level.

4.3. Deterministic forecasting

4.3.1. Performance evaluation of optimized statistical models

To evaluate the performance of statistical models in predicting the monthly COP, six promising statistical models, namely ARIMA, ARFIMA, ETS, TBAT, Theta, and Naïve are considered. The most parsimonious model and its associated parameters are obtained using the Forecast package of R [45,46]. Table 7 presents the RMSE, SMAPE, MAE, and MASE obtained using the optimized statistical models. It can be observed from Table 7 that the optimized ARIMA model provides the best RMSE, SMAPE, MAE, and MASE than all other optimized statistical models considered in this study.

4.3.2. Performance evaluation of machine learning models

In the literature, various ML models have been used for predicting the monthly COP. Therefore, in this subsection, we have considered 13 ML models (namely, linear regression, ridge regression, Huber regression, AdaBoost, random forest, gradient boosting, linear support vector regressor (SVR), multilayer perceptron (MLP), SVR, extra tree regression, bagging regression, decision tree, extreme gradient boosting (XGBoost)) in predicting the monthly COP. In addition to these 13 models, we have also implemented four ML models, namely lasso regression, elastic net regression, SGD regression, and Tweedie regression for predicting the monthly COP. However, these four models are excluded from the comparison due to significantly inferior performance to the earlier 13 ML models. Since some of the ML models are stochastic, we have repeated the simulations 50 independent times and considered the best 20 simulations based on the RMSE measure for comparison. The mean and standard deviation (std. dev.) of the best 20 simulations are

presented in Table 8. It can be observed from Table 8 that the Huber regression model provides the best forecasting accuracies among all the ML models. Therefore, to check the statistical significance, the Wilcoxon Signed-Rank Test (WSRT) is applied with a 95 % significance level, and the results are presented in Table 9. It can be observed from Table 9 that the Huber regression model provides statistically superior performance than the 12 comparative ML models in all four accuracy measures.

4.3.3. Performance evaluation of optimized DNN models

In this study, five DL models namely, LSTM, BiLSTM, CNN, GRU, and ConvLSTM are optimized using the proposed DE-DL method for predicting the monthly COP. Since the DL models are stochastic, 50 independent simulations are conducted and the best 20 results based on RMSE are considered for comparison. Table 10 presents the mean and standard deviation (Std. Dev.) of the best 20 simulations. It can be observed that the optimized BiLSTM model provides the best mean RMSE, SMAPE, MAE, and MASE than its counterparts. Additionally, the WSRT is applied, and the results are presented in Table 11. It can be observed from Table 11 that the optimized BiLSTM model provides statistically better RMSE, SMAPE, MAE, and MASE than optimized LSTM, CNN, and ConvLSTM models. However, the optimized GRU model provides statistically equivalent MAE and MASE to the optimized BiLSTM model.

4.3.4. Performance evaluation of optimized additive hybrid models

To evaluate the performance of additive hybrid models, we have considered an optimized ARIMA model to capture the linear component of monthly COP data and used optimized DL models to capture the nonlinear component existing in the residual series. We have considered ARIMA in the hybrid model because the optimized ARIMA model outperforms other statistical models in predicting the monthly COP data (as in Table 7). Five additive hybrid models are obtained (aARIMA-LSTM, aARIMA-BiLSTM, aARIMA-CNN, aARIMA-GRU, aARIMA-ConvLSTM) by employing optimized ARIMA to model the linear component and optimized DL models (LSTM, BiLSTM, CNN, GRU and ConvLSTM) to model the nonlinear component. Table 12 presents the mean and standard deviation of the best 20 independent simulations based on RMSE. It can be observed from Table 12 that the aARIMA-GRU model provides the best RMSE, SMAPE, MAE, and MASE than its counterparts. Since the DL models are stochastic, we have applied the WSRT and presented the test results in Table 13. It can be observed from Table 13 that the aARIMA-GRU model statistically outperforms other additive hybrid models in predicting the monthly COP employing RMSE, SMAPE, MAE, and MASE measures.

4.3.5. Performance evaluation of optimized multiplicative hybrid models

To evaluate the performance of multiplicative hybrid models, we have considered an optimized ARIMA model to capture the linear component of monthly COP data and used optimized DL models to capture the nonlinear component existing in the residual series. We have considered ARIMA in the hybrid model because the optimized ARIMA model outperforms other statistical models in predicting the monthly COP data (as in Table 7). Five multiplicative hybrid models are obtained (mARIMA-LSTM, mARIMA-BiLSTM, mARIMA-CNN, mARIMA-GRU, mARIMA-ConvLSTM) by employing optimized ARIMA to model the linear component and optimized DL models (LSTM, BiLSTM, CNN, GRU and ConvLSTM) to model the nonlinear component. Table 14 presents the mean and standard deviation of the best 20 independent simulations based on RMSE. It can be observed from Table 14 that the mARIMA-GRU model provides the best RMSE, and mARIMA-LSTM provides the best SMAPE, MAE, and MASE than its counterparts. Since the DL models are stochastic, we have applied the WSRT and presented the test results in Table 15. It can be observed from Table 15 that the mARIMA-GRU model statistically outperforms mARIMA-BiLSTM, mARIMA-CNN, mARIMA-ConvLSTM in predicting the monthly COP employing RMSE, SMAPE, MAE, and MASE measures. However, the mARIMA-GRU model is statistically equivalent to mARIMA-LSTM in MAE and MASE measures.

4.3.6. Overall ranking of forecasting models using deterministic forecasts

In this subsection, the best optimized statistical model (ARIMA), best ML model (Huber Regression), best optimized DL model (BiLSTM), best additive hybrid model (aARIMA-GRU) and best multiplicative hybrid model (mARIMA-GRU) are ranked based on deterministic forecasts. The best 20 independent simulations are considered for ranking the models using the Friedman and Nemenyi Hypothesis Test (FNHT) with a 95 % significance level. The test results based on RMSE, SMAPE, MAE, and MASE are presented in Fig. 7. It can be observed from Fig. 7 that the aARIMA-GRU model has the lowest mean rank (means the best rank) among all the models considering RMSE, MAE and MASE measures. Although the BiLSTM model has the lowest mean rank in the SMAPE measure, the aARIMA-GRU model is statistically equivalent to BiLSTM (since the difference in mean rank is less than the critical distance 6.1). Although the BiLSTM model is statistically equivalent to the aARIMA-GRU model in SMAPE, MAE, and MASE measures, the BiLSTM model provides a statistically inferior rank in the RMSE measure. Therefore, the aARIMA-GRU is the statistically superior model in

Table 7
Forecasting Accuracies of Optimized Statistical Models in Predicting Monthly COP Data (Best values are denoted in bold face).

	RMSE	SMAPE	MAE	MASE
ARIMA	**5.7027**	**8.5671**	**4.3607**	**0.9772**
ARFIMA	5.7274	8.5954	4.3720	0.9797
ETS	5.9609	9.2684	4.5499	1.0196
TBAT	5.8389	8.5319	4.4709	1.0019
THETA	5.8442	8.9199	4.4874	1.0056
NAÏVE	5.8440	8.9197	4.4874	1.0056

Table 8
Forecasting Accuracies of ML Models in Predicting Monthly COP Data (Best values are denoted in bold face).

	RMSE Mean ± Std. Dev.	SMAPE Mean ± Std. Dev.	MAE Mean ± Std. Dev.	MASE Mean ± Std. Dev.
Linear Regression	5.925 ± 1.823E-15	9.297 ± 3.64501E-15	4.622 ± 9.11E-16	1.036 ± 0
Ridge Regression	7.252 ± 3.645E-15	10.409 ± 1.8225E-15	5.398 ± 0.00E + 00	1.210 ± 2.278E-16
Hubber Regression	**5.729 ± 2.734E-15**	**8.902 ± 1.8225E-15**	**4.462 ± 1.82E-15**	**1.000 ± 1.139E-16**
AdaBoost	6.705 ± 1.485E-01	9.423 ± 0.248	4.830 ± 1.10E-01	1.082 ± 0.0246
Random Forest	6.540 ± 5.010E-02	9.591 ± 0.107	4.843 ± 5.72E-02	1.085 ± 0.0128
Gradient Boosting	6.642 ± 1.236E-02	9.528 ± 0.040	4.902 ± 1.91E-02	1.099 ± 0.0042
Linear SVR	5.805 ± 4.945E-03	8.796 ± 0.036	4.415 ± 1.56E-02	0.989 ± 0.0035
MLP	8.650 ± 5.822E-01	12.020 ± 0.640	6.314 ± 4.04E-01	1.415 ± 0.0904
SVR	10.376 ± 3.645E-15	14.705 ± 3.64501E-15	7.635 ± 9.11E-16	1.711 ± 2.278E-16
ExtraTrees Regression	8.647 ± 1.184E-01	12.362 ± 0.235	6.425 ± 1.13E-01	1.440 ± 0.0253
Bagging Regression	6.582 ± 1.723E-01	9.719 ± 0.351	4.913 ± 1.66E-01	1.101 ± 0.0371
Decision Tree	8.049 ± 1.342E-01	12.388 ± 0.357	6.252 ± 1.70E-01	1.401 ± 0.0380
XGBoost	6.886 ± 0.000E + 00	9.941 ± 3.64501E-15	5.200 ± 9.11E-16	1.165 ± 2.278E-16

Table 9
Wilcoxon Signed-Rank Test results indicating the worse (−), better (+), and equivalent (≈) ML model than Huber Regression in Predicting Monthly COP Data.

	RMSE	SMAPE	MAE	MASE
Linear Regression	−	−	−	−
Ridge Regression	−	−	−	−
AdaBoost	−	−	−	−
Random Forest	−	−	−	−
Gradient Boosting	−	−	−	−
Linear SVR	−	−	−	−
MLP	−	−	−	−
SVR	−	−	−	−
ExtraTrees Regression	−	−	−	−
Bagging Regression	−	−	−	−
Decision Tree	−	−	−	−
XGBoost	−	−	−	−

Table 10
Forecasting Accuracies of Optimized Deep Learning Models in Predicting Monthly COP Data (Best values are denoted in bold face).

	RMSE Mean ± Std. Dev.	SMAPE Mean ± Std. Dev.	MAE Mean ± Std. Dev.	MASE Mean ± Std. Dev.
LSTM	6.439 ± 0.259	9.270 ± 0.360	4.657 ± 0.155	1.044 ± 0.035
BiLSTM	**5.586 ± 0.041**	**8.275 ± 0.194**	**4.235 ± 0.078**	**0.949 ± 0.017**
CNN	5.967 ± 0.104	8.755 ± 0260	4.505 ± 0.106	1.009 ± 0.024
GRU	5.675 ± 0.090	8.429 ± 0.215	4.278 ± 0.095	0.959 ± 0.021
ConvLSTM	5.809 ± 0.038	8.663 ± 0.211	4.419 ± 0.090	0.990 ± 0.020

predicting the monthly COP than all other models considered in this study and is used for probabilistic forecasting of COP.

For a visual understanding of the performance of the best models in predicting monthly COP, we have considered the best simulation out of 50 independent simulations based on RMSE measure and plotted the comparison graph in Fig. 8. It can be observed from Fig. 8 that the aARIMA-GRU model shows better fitting to true values (original observations) of the monthly COP time series. Further, to observe the histogram and distribution fitting of true values and aARIMA-GRU best forecasts, we have plotted the joint plot in Fig. 9. It can be observed from Fig. 9 that the histogram and distribution of true COP values and aARIMA-GRU forecasts are nearly

Table 11
Wilcoxon Signed-Rank Test results indicating the worse (−), better (+) and equivalent (≈) optimized deep learning model than BiLSTM in Predicting Monthly COP Data.

	RMSE	SMAPE	MAE	MASE
LSTM	−	−	−	−
CNN	−	−	−	−
GRU	−	−	≈	≈
ConvLSTM	−	−	−	−

Table 12
Forecasting Accuracies of Additive Hybrid Models employing Optimized Deep Learning Models and Optimized ARIMA Model in Predicting Monthly COP Data (Best values are denoted in bold face).

	RMSE Mean ± Std. Dev.	SMAPE Mean ± Std. Dev.	MAE Mean ± Std. Dev.	MASE Mean ± Std. Dev.
aARIMA-LSTM	5.645 ± 0.027	8.473 ± 0.129	4.303 ± 0.046	0.964 ± 0.010
aARIMA-BiLSTM	5.767 ± 0.024	8.478 ± 0.136	4.337 ± 0.052	0.972 ± 0.012
aARIMA-CNN	5.934 ± 0.027	9.034 ± 0.081	4.561 ± 0.032	1.022 ± 0.007
aARIMA-GRU	**5.519 ± 0.021**	**8.333 ± 0.124**	**4.227 ± 0.035**	**0.947 ± 0.008**
aARIMA-ConvLSTM	5.858 ± 0.012	8.918 ± 0.078	4.520 ± 0.025	1.013 ± 0.006

Table 13
Wilcoxon Signed-Rank Test results indicating the worse (−), better (+) and equivalent (≈) Additive Hybrid Models than aARIMA-GRU in Predicting Monthly COP Data.

	RMSE	SMAPE	MAE	MASE
aARIMA-LSTM	−	−	−	−
aARIMA-BiLSTM	−	−	−	−
aARIMA-CNN	−	−	−	−
aARIMA-ConvLSTM	−	−	−	−

Table 14
Forecasting Accuracies of Multiplicative Hybrid Models employing Optimized Deep Learning Models and Optimized ARIMA Model in Predicting Monthly COP Data (Best values are denoted in bold face).

	RMSE Mean ± Std. Dev.	SMAPE Mean ± Std. Dev.	MAE Mean ± Std. Dev.	MASE Mean ± Std. Dev.
mARIMA-LSTM	5.653 ± 0.043	**8.448 ± 0.100**	**4.282 ± 0.045**	**0.096 ± 0.010**
mARIMA-BiLSTM	5.680 ± 0.027	8.657 ± 0.106	4.386 ± 0.054	0.983 ± 0.012
mARIMA-CNN	6.035 ± 0.030	9.104 ± 0.169	4.636 ± 0.068	1.039 ± 0.015
mARIMA-GRU	**5.626 ± 0.023**	8.515 ± 0.119	4.290 ± 0.058	0.961 ± 0.013
mARIMA-ConvLSTM	5.857 ± 0.012	8.799 ± 0.080	4.479 ± 0.037	1.004 ± 0.008

Table 15
Wilcoxon Signed-Rank Test results indicating the worse (−), better (+), and equivalent (≈) Multiplicative Hybrid Model than mARIMA-GRU in Predicting Monthly COP Data.

	RMSE	SMAPE	MAE	MASE
mARIMA-LSTM	−	+	≈	≈
mARIMA-BiLSTM	−	−	−	−
mARIMA-CNN	−	−	−	−
mARIMA-ConvLSTM	−	−	−	−

same which again shows the promising quality of the aARIMA-GRU model optimized with the proposed DE-DL method and Forecast package of R.

4.4. Probabilistic forecasting

It is inevitable to have prediction errors around the deterministic forecasts of COP time series since the forecasting of COP is affected by many factors, such as macroeconomic factors, geopolitical events, supply and demand dynamics, choice of forecasting method, employed model, and its associated parameters. Due to these uncertainties making decisions related to COP becomes more

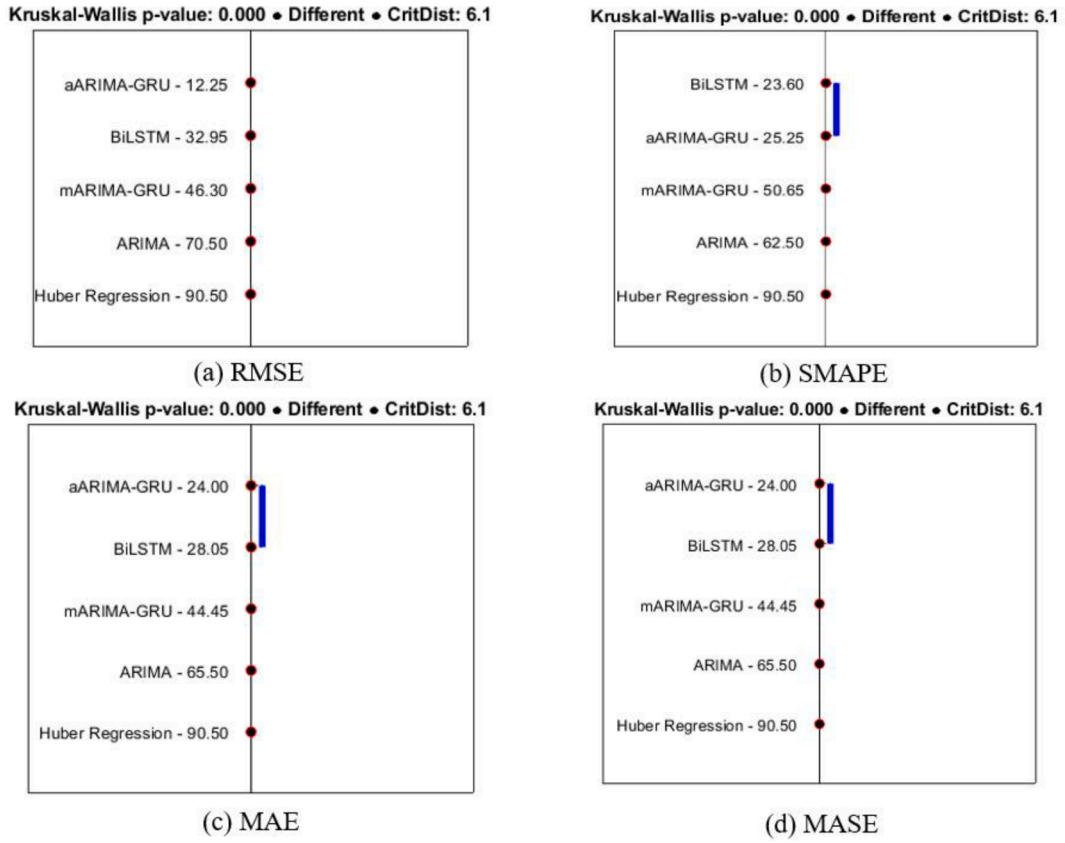

Fig. 7. Mean Rank of Forecasting Models employing Friedman and Nemenyi Hypothesis Test on (a) RMSE, (b) SMAPE, (c) MAE, and (d) MASE measures.

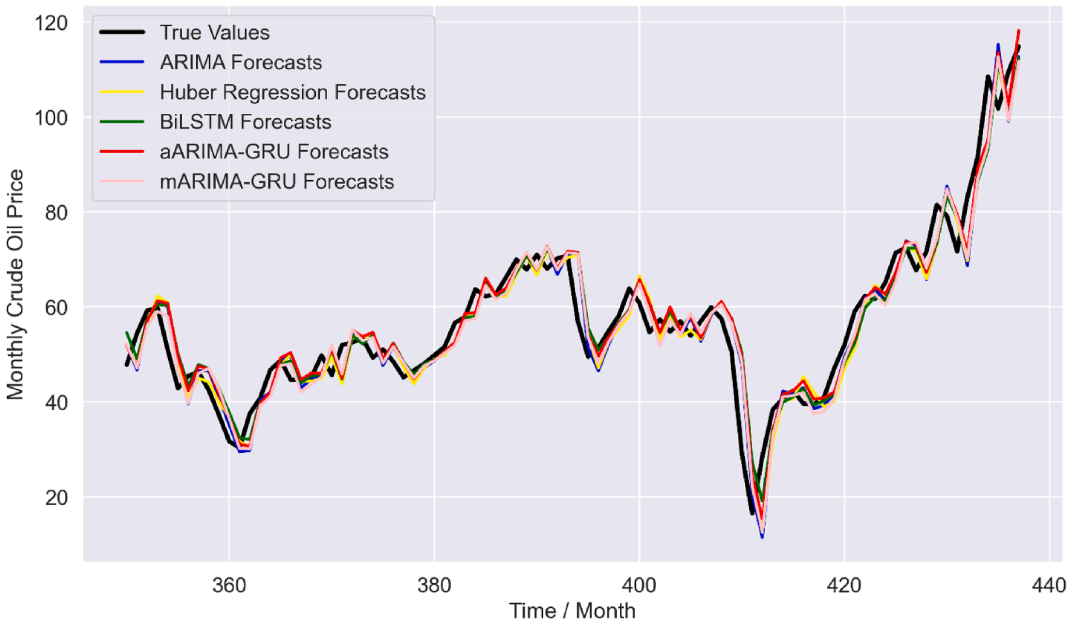

Fig. 8. Comparison of True Values (Observed) and Best Forecasts using the Best Models.

challenging and most often adversely affects risk management. Hence, quantifying uncertainties plays a vital role in financial risk management. In this paper, to efficiently quantify the uncertainties, the statistical properties of predictive errors obtained using the best deterministic forecasts from the optimized aARIMA-GRU model are carefully analyzed. The Gaussian distribution and t Location Scale distribution are fitted to the predictive errors (obtained by subtracting optimized aARIMA-GRU deterministic forecasts from the COP time series), and the distribution fitting of predictive errors on the train set is presented in Fig. 10. It can be observed from Fig. 10 that the t Location Scale distribution has better fitted the predictive errors.

Once the uncertainty analysis is over, the prediction intervals for the COP can be computed by employing the fitted distributions with the best location and scale value ($Best_{\propto/2}$) using Eq. (10), where $\propto$ denotes the significance level, $U^{\propto}$ and $L^{\propto}$ denotes the upper and lower bound of the constructed interval at $\propto$-significance level, $y^{'}$ denotes the forecasted values with the best RMSE from all the independent runs using the optimized aARIMA-GRU model, e is the fitted errors, and $var(\bullet)$ denotes the variance calculating function.

$$[L^{\propto}, U^{\propto}] = \left[y^{'} - Best_{\propto/2}\sqrt{var(e)}\,, y^{'} + Best_{\propto/2}\sqrt{var(e)} \right] \tag{10}$$

The prediction intervals with 90 %, 80 %, 70 %, and 60 % confidence levels can be computed by using Eq. (10). Fig. 11 presents the prediction intervals obtained using the optimal Gaussian distribution and the t Location Scale distribution. The near-optimal location is 0, and the scale is 5 for both distributions. It can be visualized from Fig. 11 that both distributions are generating similar prediction intervals. However, to distinguish the two alternatives, quantitative uncertainty metrics, such as PICP, PINAW, AWD, and ACE, of the obtained prediction intervals are computed and presented in Table 16.

It can be observed from Table 16 that the PICP of Gaussian and t Location Scale for 90 %, 80 %, and 70 % confidence levels are the same. In comparison, the Gaussian distribution has a lower PICP than the t Location Scale distribution with a 60 % confidence level. The Gaussian distribution has a slightly smaller PINAW than the t Location Scale in 90 % and 80 % confidence levels while the Gaussian distribution has the same PINAW value as the t Location Scale distribution in 70 % and 60 % confidence levels. The Gaussian distribution has higher AWD than the t Location Scale distribution in all confidence levels. The Gaussian distribution provides positive ACE in 90 %, 80 %, and 70 % confidence levels, while the t Location Scale distribution provides positive ACE in all confidence levels. The goal of probabilistic forecasting is to obtain prediction intervals with desired PICP (reliability) and PINAW (resolution). On the other hand, a trade-off should be maintained between reliability and resolution. For this, AWD and ACE measures are used to integrate reliability and resolution. A desired prediction interval should have higher PICP, lower PINAW, lower AWD, and positive ACE. From Table 16, it can be observed that the t Location Scale distribution provides better prediction intervals than the Gaussian distribution since it provides better mean ACE (higher) and AWD (lower) than the Gaussian distribution.

5. Discussion

This section presents the overall discussion of the results obtained from this study. In simulations, six optimized statistical models, five optimized DL models, five optimized additive hybrid models, five optimized multiplicative hybrid models, and 13 ML models are employed for deterministic forecasting of the COP time series. The optimized statistical models used in this paper are deterministic and hence, don't have any uncertainty associated with the computation of model parameters and forecasts. However, the DL models, additive and multiplicative hybrid models are stochastic and have uncertainty in both model parameter estimation and providing forecasts for monthly COP. To reduce the uncertainties associated with the DL model parameters, the architecture and other hyperparameters of the DL models (number of layers, number of neurons in each layer, activation function, kernel initializer, dropout rate, batch size, and learning rate) are optimized using the proposed DE-DL method and trained the DL models using the most promising training algorithm "adam". Non-parametric statistical tests such as "Wilcoxon Signed-Rank Test" and "Friedman and Nemenyi Hypothesis Test" are employed with a 95 % confidence level on the obtained 20 best results from 50 independent simulations to draw reliable conclusions and reduce the uncertainties. Furthermore, to model the uncertainties due to the choice of a forecasting method, employed model and its associated parameters, in this paper, probabilistic forecasts are computed and quantitative uncertainty methods are used to determine the reliability of computed prediction intervals using Gaussian and t Location Scale distribution with different significance levels. Based on this research, this paper can help to reliably address the following research questions relating to forecasting monthly COP.

RQ-1: It is clear from Table 10 that the BiLSTM model optimized with the proposed DE-DL method provides the best RMSE, SMAPE, MAE, and MASE than optimized LSTM, CNN, GRU, and ConvLSTM models. Since the DL models are stochastic, the non-parametric WSRT is conducted and the results are presented in Table 11. Table 11 confirms that the optimized BiLSTM model provides statistically superior performance than optimized LSTM, CNN, and ConvLSTM in all four deterministic performance measures. The optimized BiLSTM model provides statistically better RMSE and SMAPE than the optimized GRU model. However, the optimized GRU model provides statistically equivalent MAE and MASE to the optimized BiLSTM model. Hence, the optimized BiLSTM model is the right alternative DL model for forecasting monthly COP. Additionally, since BiLSTM is computationally expensive than GRU and the optimized GRU provides statistically equivalent results to the optimized BiLSTM model, one can also employ the optimized GRU model instead of optimized LSTM, CNN, and ConvLSTM.

RQ-2: It can be concluded from Table 7 that the ARIMA model optimized with Forecast Package of R is the right alternative for forecasting monthly COP than the optimized ARFIMA, ETS, TBAT, Theta, and Naïve models.

RQ-3: It is clear from Table 13 that the additive hybrid model aARIMA-GRU employing optimized ARIMA and GRU models provides statistically the best RMSE, SMAPE, MAE, and MASE than optimized aARIMA-LSTM, aARIMA-BiLSTM, aARIMA-CNN and

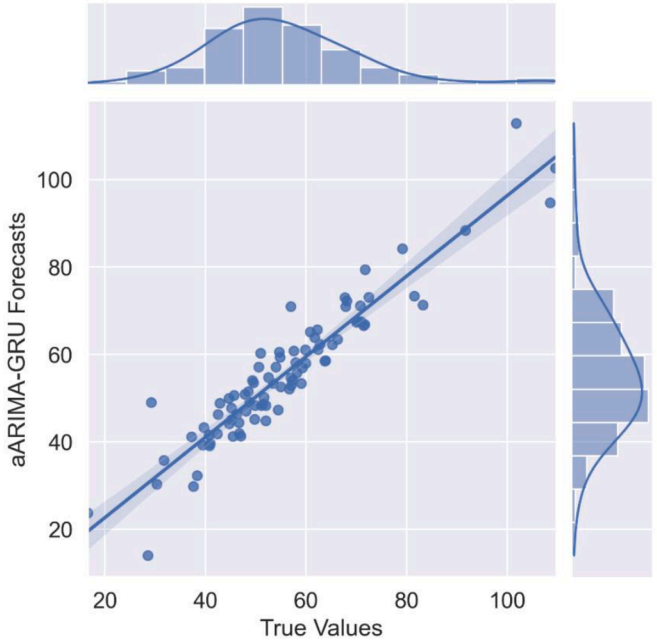

Fig. 9. Joint Plot between True COP Values and Optimized aARIMA-GRU Best Forecasts.

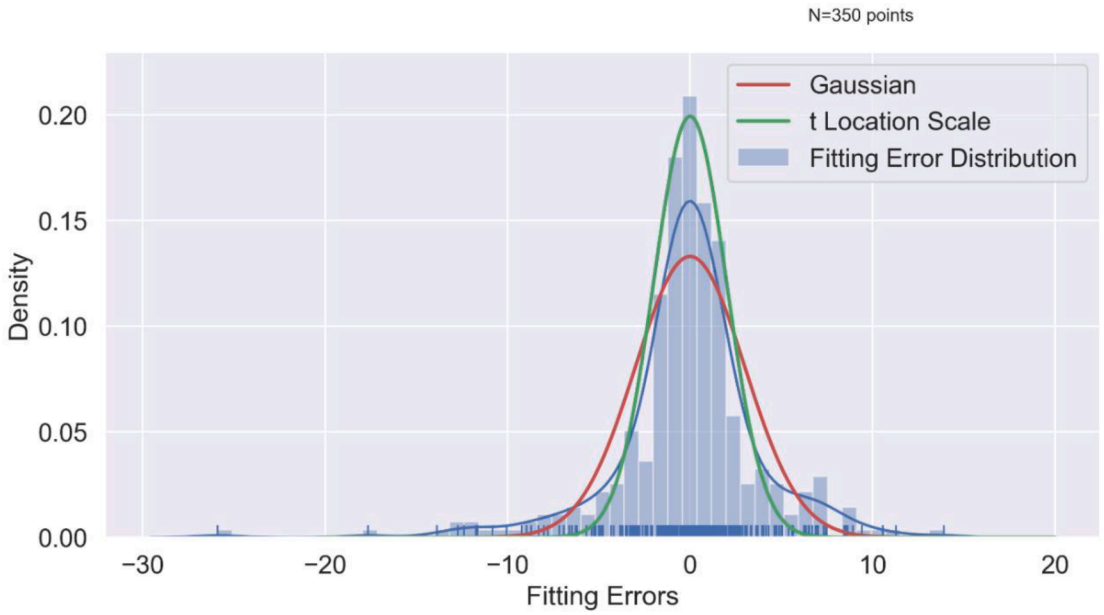

Fig. 10. Distribution fitting of the predictive errors.

aARIMA-ConvLSTM models.

RQ-4: It can be concluded from Table 15 that the multiplicative hybrid model mARIMA-GRU employing the optimized ARIMA and GRU models provides statistically better RMSE, SMAPE, MAE and MASE than mARIMA-BiLSTM, mARIMA-CNN, mARIMA-ConvLSTM. The optimized mARIMA-LSTM method provides statistically equivalent MAE and MASE than the aARIMA-GRU method. Since the LSTM model is more computationally expensive than the GRU model, the mARIMA-GRU is considered the right alternative for monthly COP forecasting among all the optimized multiplicative hybrid models considered in this paper.

RQ-5: In this study, Fig. 7 depicts that the additive hybrid model aARIMA-GRU employing optimized GRU and ARIMA models holds the Rank-1 with statistical significance among all other alternatives considered in this study. The optimized BiLSTM model holds the second-best rank. The mARIMA-GRU holds the third-best rank, while optimized ARIMA and Huber regression hold the fourth and

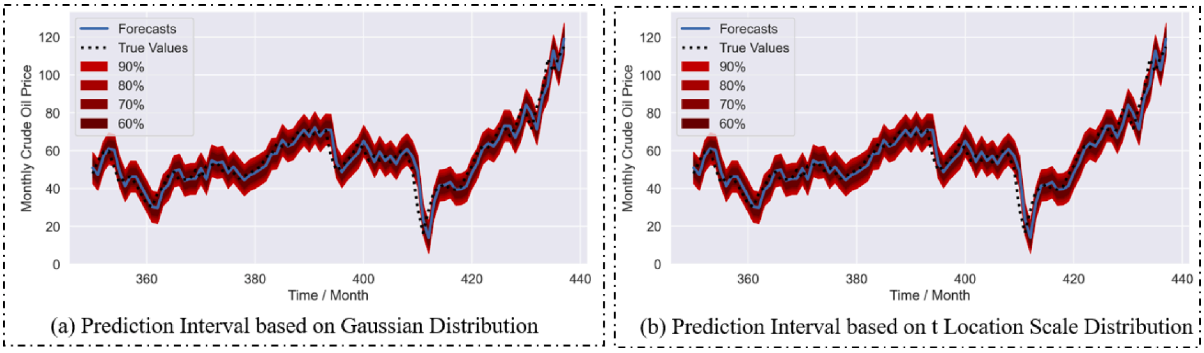

Fig. 11. Prediction interval of monthly COP using (a) Gaussian Distribution (b) t Location Scale Distribution.

Table 16
Performance evaluation of constructed prediction intervals using Gaussian distribution and t Location Scale distribution.

Confidence Level	Gaussian Distribution				t-Location Scale Distribution			
	PICP	PINAW	AWD	ACE	PICP	PINAW	AWD	ACE
90 %	92.045	0.167	0.026	2.045	92.045	0.169	0.025	2.045
80 %	82.954	0.130	0.051	2.954	82.954	0.131	0.049	2.954
70 %	73.863	0.106	0.087	3.864	73.863	0.106	0.085	3.864
60 %	59.091	0.086	0.145	−0.909	60.227	0.086	0.144	0.227
Mean	76.988	**0.122**	0.077	1.989	**77.272**	0.123	**0.076**	**2.272**

fifth ranks, respectively.

RQ-6: In this study, according to Fig. 10, the t Location Scale distribution function fits the error (generated by the optimized aARIMA-GRU best forecasts) more closely than the Gaussian distribution. It is also confirmed from Fig. 11 and Table 16. From Table 16, one can see that the mean ACE obtained using the t Location Scale distribution is more than that of the Gaussian distribution. Hence, it can be concluded that the t Location Scale distribution function is the right alternative in modeling the errors and computing the prediction intervals with 90 %, 80 %, 70 %, and 60 % significance levels.

RQ-7: The prediction intervals obtained using the t Location Scale distribution on the optimized aARIMA-GRU forecast errors are valuable and reliable since the ACE (as in Table 16) is positive for 90 %, 80 %, 70 %, and 60 % confidence levels.

RQ-8: The additive hybrid model aARIMA-GRU employing the optimized ARIMA and GRU model provides statistically superior forecasts than all other methods employed in this study. The aARIMA-GRU model uses the ARIMA (1,1,0) model (optimized and obtained using the Forecast package of R [45,46]) and GRU model (optimized using the proposed DE-DL method and presented in Table 5) to forecast the monthly COP. Therefore, the computational complexity of the proposed aARIMA-GRU model is presented. The computational complexity for training an ARIMA (p, d, q) model is $O(n.(p^2+q^2))$ and making predictions is typically linear in the number of forecasted time steps. The computational complexity of a 2-hidden layer GRU model with N_s input samples, H GRU units in the first hidden layer, F neurons in the second hidden layer and N_o number of outputs is $O(N_s(\mathrm{H}^2+\mathrm{HF}+\mathrm{F}N_o))$ [49]. The time complexity of the Adam training algorithm is $O(\mathrm{B.E.N})$ with B denoting the batch size, E denoting the number of epochs, and N denoting the number of parameters. The time complexity of the DE optimization algorithm is $O(\max\_gen.P_{size}.comp\_fitness)$ with comp_fitness denoting the computational complexity to evaluate the fitness of each decision vector. In the aARIMA-GRU hybrid method, first, the optimized ARIMA is applied to the COP time series to obtain the linear model forecasts. Then the residual series is computed by subtracting the ARIMA forecasts from the original COP time series which has a time complexity of $O(n)$. Then the residual series is modelled by the proposed DE-DL method employing the Adam algorithm to optimize the GRU architecture and other hyper-parameters. It has a time complexity of $O(\max\_gen.P_{size}.B.E.N_s(\mathrm{H}^2+\mathrm{HF}+\mathrm{F}N_o))$. Then the final forecasts are computed by adding the optimized ARIMA forecasts with the optimized GRU forecasts which have a time complexity of $O(n)$. Considering all, the proposed aARIMA-GRU employing optimized ARIMA and optimized GRU (optimized using proposed DE-DL method) models has a time complexity of $O(n.(p^2+q^2))+O(n)+\max\_gen.P_{size}.B.E.N_s(\mathrm{H}^2+\mathrm{HF}+\mathrm{F}N_o)+O(n))$ which is equivalent to $O(n.(p^2+q^2)+\max\_gen.P_{size}.B.E.N_s(\mathrm{H}^2+\mathrm{HF}+\mathrm{F}N_o))$.

6. Conclusion

This paper proposes deterministic and probabilistic forecasting methods for COP employing optimized DL, statistical, and hybrid (additive and multiplicative) models. The statistical models are optimized using the Forecast package of R, while the DL model architecture and other hyper-parameters are optimized using the proposed DE-DL method. Six optimized statistical models, five optimized DL models, five optimized additive hybrid models, five optimized multiplicative hybrid models, and 13 ML models are used to

forecast the COP data. The "Wilcoxon Signed-Rank Test" and "Friedman and Nemenyi Hypothesis Test" are applied to the obtained results for drawing decisive and reliable conclusions by modeling the uncertainties. From the extensive statistical analysis, it is concluded that the aARIMA-GRU model with optimized hyper-parameters achieves the first rank in deterministic forecasting of monthly COP considering RMSE, SMAPE, MAE, and MASE accuracy measures. Further, the best forecasts from the aARIMA-GRU model are used to compute the probabilistic forecasts employing Gaussian and t Location Scale distribution functions. Simulation results confirm that the use of t Location Scale distribution is the right alternative for reliable probabilistic forecasting (prediction interval) of monthly COP using PICP, PINAW, AWD, and ACE performance measures with 90 %, 80 %, 70 %, and 60 % confidence levels.

The proposed optimized aARIMA-GRU hybrid model provides accurate and reliable point and interval forecasts for the monthly COP. Therefore, it has great potential for risk management in crude oil markets, thereby reducing crude oil transaction costs. However, the present model only considers the quantitative historical data related to COP to predict future values. The efficiency of the proposed model can further be improved by employing some optimized model for modeling the qualitative data (Google trends, News headlines, etc.) along with it. In the future, one can employ the proposed DE-DL method to optimize the DL models for modeling the individual intrinsic mode functions generated by decomposition-based hybrid methods [12,24,25,27] and apply it to COP forecasting. Furthermore, the daily and weekly COP information can be incorporated into the monthly COP modeling to update the monthly forecasts on a daily and weekly basis.

The impact of this paper is significant since the proposed DE-DL method can be applied to optimize any DL model for any supervised regression [12,14–28,43,44,9] and classification [11,29–36,38–42,45–47] problem. Additionally, the developed optimized deterministic and probabilistic forecasting methods can be used in forecasting other time series data like electricity load, electricity price, wind speed, stock price, internet traffic, telecom traffic, sunspot number, air quality index, call volumes in a call center, demand of a specific product, temperature, rainfall, etc.

CRediT authorship contribution statement

Sourav Kumar Purohit: Writing – original draft, Methodology, Software, Validation, Investigation, Resources, Data curation. **Sibarama Panigrahi:** Conceptualization, Investigation, Methodology, Software, Formal analysis, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This research work was catalyzed and supported by the Science and Engineering Research Board (SERB), DST, Government of India under the Core Research Grant (CRG) scheme with Grant No. CRG/2021/006122.

References

[1] J.L. Zhang, Y.J. Zhang, L. Zhang, A novel hybrid method for crude oil price forecasting, Energy Econ. 49 (2015) 649–659.
[2] L. Yu, W. Dai, L. Tang, J. Wu, A hybrid grid-GA-based LSSVR learning paradigm for crude oil price forecasting, Neural Comput. Applic. 27 (8) (2016) 2193–2215.
[3] N.B. Behmiri, J.R.P. Manso, How crude oil consumption impacts on economic growth of Sub-Saharan Africa? Energy 54 (2013) 74–83.
[4] T. Cavalcanti, J.T. Jalles, Macroeconomic effects of oil price shocks in Brazil and in the United States, Appl. Energy 104 (2013) 475–486.
[5] Y.B. Özçelik, A. Altan, Overcoming Nonlinear Dynamics in Diabetic Retinopathy Classification: A Robust AI-Based Model with Chaotic Swarm Intelligence Optimization and Recurrent Long Short-Term Memory, Fractal and Fractional 7 (8) (2023) 598.
[6] S. Karasu, A. Altan, Crude oil time series prediction model based on LSTM network with chaotic Henry gas solubility optimization, Energy 242 (2022), 122964.
[7] Z. Jiang, L. Zhang, L. Zhang, B. Wen, Investor sentiment and machine learning: Predicting the price of China's crude oil futures market, Energy 247 (2022), 123471.
[8] S. Urolagin, N. Sharma, T.K. Datta, A combined architecture of multivariate LSTM with Mahalanobis and Z-Score transformations for oil price forecasting, Energy 231 (2021), 120963.
[9] Y. Li, S. Jiang, X. Li, S. Wang, The role of news sentiment in oil futures returns and volatility forecasting: Data-decomposition based deep learning approach, Energy Econ. 105140 (2021).
[10] G.A. Busari, D.H. Lim, Crude oil price prediction: A comparison between AdaBoost-LSTM and AdaBoost-GRU for improving forecasting performance, Comput. Chem. Eng. 155 (2021), 107513.
[11] B. Wu, L. Wang, S. Wang, Y.R. Zeng, Forecasting the US oil markets based on social media information during the COVID-19 pandemic, Energy 226 (2021), 120403.
[12] Z. Hajiabotorabi, F.F. Samavati, F.M.M. Ghaini, A. Shahmoradi, Multi-WRNN model for pricing the crude oil futures market, Expert Syst. Appl. 182 (2021), 115229.
[13] H. Jiang, W. Hu, L. Xiao, Y. Dong, A decomposition ensemble based deep learning approach for crude oil price forecasting, Resour. Policy 78 (2022), 102855.

[14] A.A. Salamai, Deep learning framework for predictive modeling of crude oil price for sustainable management in oil markets, Expert Syst. Appl. 211 (2023), 118658.
[15] S. Zhang, J. Luo, S. Wang, F. Liu, Oil price forecasting: A hybrid GRU neural network based on decomposition–reconstruction methods, Expert Syst. Appl. 218 (2023), 119617.
[16] B. Wang, J. Wang, Energy futures and spots prices forecasting by hybrid SW-GRU with EMD and error evaluation, Energy Econ. 90 (2020), 104827.
[17] J. Wang, T. Niu, P. Du, W. Yang, Ensemble probabilistic prediction approach for modeling uncertainty in crude oil price, Appl. Soft Comput. 95 (2020), 106509.
[18] Y. Zou, L. Yu, G.K. Tso, K. He, Risk forecasting in the crude oil market: A multiscale Convolutional Neural Network approach, Physica A 541 (2020), 123360.
[19] Y. Huang, Y. Deng, A new crude oil price forecasting model based on variational mode decomposition, Knowl.-Based Syst. 213 (2021), 106669.
[20] M. Srivastava, A. Rao, J.S. Parihar, S. Chavriya, S. Singh, What do the AI methods tell us about predicting price volatility of key natural resources: Evidence from hyperparameter tuning, Resour. Policy 80 (2023), 103249.
[21] N. Bacanin, L. Jovanovic, M. Zivkovic, V. Kandasamy, M. Antonijevic, M. Deveci, I. Strumberger, Multivariate energy forecasting via metaheuristic tuned long-short term memory and gated recurrent unit neural networks, Inf. Sci. 642 (2023), 119122.
[22] J. Bi, L. Zhang, H. Yuan, J. Zhang, Multi-indicator water quality prediction with attention-assisted bidirectional LSTM and encoder-decoder, Inf. Sci. 625 (2023) 65–80.
[23] C.K. Mbamba, D.J. Batstone, Optimization of deep learning models for forecasting performance in the water industry using genetic algorithms, Comput. Chem. Eng. 175 (2023), 108276.
[24] S. Sarangi, P.K. Dash, R. Bisoi, Probabilistic prediction of wind speed using an integrated deep belief network optimized by a hybrid multi-objective particle swarm algorithm, Eng. Appl. Artif. Intel. 126 (2023), 107034.
[25] B. Gülmez, Stock price prediction with optimized deep LSTM network with artificial rabbits optimization algorithm, Expert Syst. Appl. 227 (2023), 120346.
[26] A.K. Abasi, M. Aloqaily, M. Guizani, Optimization of CNN using modified Honey Badger Algorithm for Sleep Apnea detection, Expert Syst. Appl. 229 (2023), 120484.
[27] L. Souquet, N. Shvai, A. Llanza, A. Nakib, Convolutional neural network architecture search based on fractal decomposition optimization algorithm, Expert Syst. Appl. 213 (2023), 118947.
[28] L. Wen, L. Gao, X. Li, H. Li, A new genetic algorithm based evolutionary neural architecture search for image classification, Swarm Evol. Comput. 75 (2022), 101191.
[29] K. Laddach, R. Łangowski, T.A. Rutkowski, B. Puchalski, An automatic selection of optimal recurrent neural network architecture for processes dynamics modelling purposes, Appl. Soft Comput. 116 (2022), 108375.
[30] S.M.J. Jalali, M. Ahmadian, S. Ahmadian, R. Hedjam, A. Khosravi, S. Nahavandi, X-ray image based COVID-19 detection using evolutionary deep learning approach, Expert Syst. Appl. 201 (2022), 116942.
[31] S.C. Nistor, G. Czibula, IntelliSwAS: Optimizing deep neural network architectures using a particle swarm-based approach, Expert Syst. Appl. 187 (2022), 115945.
[32] A. Ghosh, N.D. Jana, S. Mallik, Z. Zhao, Designing optimal convolutional neural network architecture using differential evolution algorithm, Patterns 3 (9) (2022), 100567.
[33] T. Hassanzadeh, D. Essam, R. Sarker, EvoDCNN: An evolutionary deep convolutional neural network for image classification, Neurocomputing 488 (2022) 271–283.
[34] R. Shang, S. Zhu, J. Ren, H. Liu, L. Jiao, Evolutionary neural architecture search based on evaluation correction and functional units, Knowl.-Based Syst. 109206 (2022).
[35] C. Zhang, H. Ma, L. Hua, W. Sun, M.S. Nazir, T. Peng, An evolutionary deep learning model based on TVFEMD, improved sine cosine algorithm, CNN and BiLSTM for wind speed prediction, Energy 124250 (2022).
[36] Y. Li, J. Liu, Y. Teng, A decomposition-based memetic neural architecture search algorithm for univariate time series forecasting, Appl. Soft Comput. 130 (2022), 109714.
[37] C. He, H. Tan, S. Huang, R. Cheng, Efficient evolutionary neural architecture search by modular inheritable crossover, Swarm Evol. Comput. 64 (2021), 100894.
[38] H. Louati, S. Bechikh, A. Louati, C.C. Hung, L.B. Said, Deep convolutional neural network architecture design as a bi-level optimization problem, Neurocomputing 439 (2021) 44–62.
[39] F.E. Fernandes Jr, G.G. Yen, Pruning deep convolutional neural networks architectures with evolution strategy, Inf. Sci. 552 (2021) 29–47.
[40] J. Wei, X. Zhang, Z. Zhuo, Z. Ji, Z. Wei, J. Li, Q. Li, Leader population learning rate schedule, Inf. Sci. 623 (2023) 455–468.
[41] P.N. Rao, S. Meher, ORG-RGRU: An automated diagnosed model for multiple diseases by heuristically based optimized deep learning using speech/voice signal, Biomed. Signal Process. Control 88 (2024), 105493.
[42] R.A.A. Viadinugroho, D. Rosadi, A Weighted Metric Scalarization Approach for Multiobjective BOHB Hyperparameter Optimization in LSTM Model for Sentiment Analysis, Inf. Sci. 119282 (2023).
[43] X. Meng, Y. Zhang, L. Quan, J. Qiao, A self-organizing fuzzy neural network with hybrid learning algorithm for nonlinear system modeling, Inf. Sci. 642 (2023), 119145.
[44] T.Y. Hong, C.C. Chen, Hyperparameter Optimization for Convolutional Neural Network by Opposite-based Particle Swarm Optimization and an Empirical Study of Photomask Defect Classification, Appl. Soft Comput. 110904 (2023).
[45] R. Hyndman, G. Athanasopoulos, C. Bergmeir, G. Caceres, L. Chhay, M. O'Hara-Wild, F. Petropoulos, S. Razbash, E. Wang, F. Yasmeen (2021). forecast: Forecasting functions for time series and linear models. Rpackage version 8.15, <URL: https://pkg.robjhyndman.com/forecast/>.
[46] R.J. Hyndman, Y. Khandakar, Automatic time series forecasting: the forecast package for R, J. Stat. Softw. 26 (3) (2008) 1–22.
[47] D.S.D.O.S. Júnior, P.S. de Mattos Neto, J.F. de Oliveira, G.D. Cavalcanti, A hybrid system based on ensemble learning to model residuals for time series forecasting, Inf. Sci. 649 (2023), 119614.
[48] T. Rawald, M. Sips, N. Marwan, PyRQA - Conducting Recurrence Quantification Analysis on Very Long Time Series Efficiently. -, Comput. Geosci. 104 (2017) 101–108.
[49] T. Zhong, Z. Wen, F. Zhou, G. Trajcevski, K. Zhang, Session-based recommendation via flow-based deep generative networks and Bayesian inference, Neurocomputing 391 (2020) 129–141.